Notas de Aula



Ariel Alves da Silva



Academic year 2026

Índice

Índice 2
Arquitetura e Organização de Computadores (SCC0902) 7
1. Primeira Parte 8
1.1. Arquitetura de Von-Neumann 8
CPU 8
Registers 8
Bus 8
1.2. Ciclo de instrução 8
Ciclo de busca 8
Ciclo de execução 9
1.3. Arquitetura RISC-V 9
Compilando .asm 9
Estrutura de código 9
Diretivas 9
Registradores 9
Comandos 9
Ecalls 10
Alocando na stack 11
Instruction Set Architecture 11
Formato das instruções (6 tipos) 12
Tipo R (register) 12
Tipo I (immediate) 12
Tipo S (Store) 12
Tipo B (branch) 12
Tipo U (Upper immediate) 12
Tipo J 13
1.4. Monociclo (RISC-V) 13
Metodologia de Clocking 13
Componentes 13
Arquitetura do banco de registradores 13

Arquitetura da ALU 14
Arquitetura de operações com memória 14
Execução de instruções 15
Execução das instruções tipo R 15
Execução de load/store 15
Execução de Branch 15
Arquitetura completa 16
Tabelas e Blocos Auxiliares 17
1.5. Notas avulsas 17
−4 em binário (20 bits) 17
Formato de números inteiros 18
2. Revisão de Lógica digital 18
2.1. Circuitos combinacionais e sequenciais 18
2.2. Identidades básicas 18
2.3. Mapas de Karnaugh 18
2.4. Flip Flops 19
Flip-Flop S-R 19
Flip-Flop J-K 19
2.5. Registradores 19
3. Segunda Parte 20
3.1. Multiciclo 20
3.2. Pipeline 20
Etapas 20
Comparação entre Monociclo, Multiciclo e Pipeline 21
Considerações 22
Registradores intermediários do pipeline 22
Dependências ou conflitos 23
Conflitos estruturais 23
Dependência de dados 23
Adiantamento de Dados (forwarding) 24
Matemática Discreta (SMA0180) 27
1. Contagem 28
2. Contagem de listas, permutações e subconjuntos 29
3. Coeficientes binomiais 30
4. Relações de equivalência e contagem 31
5. Aritmética modular 31

5.1. Anotações ... 38
6. Outros ... 40
7. RSA ... 40
Introdução à Ciência da Computação II (SCC0201) ... 42
1. Análise de Algoritmos ... 43
1.1. Notações assintóticas ... 43
1.2. Exemplo: subarranjo máximo (divisão e conquista) ... 43
1.3. Recorrências ... 44
Método da substituição ... 44
Método da árvore de recursão ... 45
Método mestre ... 45
2. Métodos de Ordenação ... 46
2.1. Bubble Sort ... 46
2.2. Insertion Sort ... 46
2.3. Selection Sort (seleção direta) ... 46
2.4. Shell Sort ... 46
2.5. Quick Sort ... 47
2.6. Heap Sort ... 47
2.7. Radix Sort ... 47
2.8. Comparação entre os métodos ... 48
3. Lista 2 - Complexidade ... 49
3.1. Ex. 1: menor n com $100n^2 < 2^n$ ... 49
3.2. Ex. 2: "no mínimo $\mathcal{O}(n^2)$" ... 49
3.3. Ex. 3: $2^{n+1} = \mathcal{O}(2^n)$? $2^{2n} = \mathcal{O}(2^n)$? ... 49
3.4. Ex. 5: $T(n) = T(n-1) + n$ é $\mathcal{O}(n^2)$ ... 49
Equações Diferenciais Ordinárias (SME0340) ... 50
1. Introdução ... 51
1.1. Problema de Valor Inicial (PVI) ... 51
2. EDOs de 1ª ordem ... 52
2.1. EDOs separáveis ... 52
2.2. EDOs lineares de 1ª ordem ... 52
2.3. EDOs não lineares de 1ª ordem ... 53
Equação de Bernoulli ... 53
Equação de Riccati ... 53
3. Aplicações ... 53
3.1. Problemas de crescimento / decaimento ... 53

3.2. Lei do resfriamento / aquecimento de Newton 54
3.3. Diluição 54
4. EDOs de 2ª ordem 54
4.1. EDOs de 2ª ordem homogêneas 54
4.2. EDOs (H, 2ª ordem) com coeficientes constantes 55
4.3. Teorema de Abel (ordem n) 56
4.4. Resolução de EDOs não homogêneas de 2ª ordem 57
5. EDOs de ordem n 57
5.1. Números complexos 58
5.2. Radiciação complexa 58
5.3. Método das raízes racionais 58
5.4. Circuito em série RLC 59
6. Sistemas de EDOs de 1ª ordem 59
6.1. O que são? 59
6.2. Solucionando sistemas $X' = AX$ (homogêneos) 61
Autovalores complexos de A 61
Autovalores repetidos 61
6.3. Sistemas lineares não homogêneos 62
6.4. EDO de ordem $k \geq 2$ como sistema 62
7. Transformada de Laplace 63
7.1. Transformadas básicas 63
7.2. Condição para convergência 63
7.3. Propriedades 64
7.4. Transformada inversa 64
7.5. Função degrau unitário 65
7.6. Função delta de Dirac 65
7.7. Convolução (importante) 66
8. Anotações da Lista 66
8.1. Como resolver $\sin()\cos()$? 66
8.2. Transformadas em cadeia 67
8.3. Interruptor (U) 67
8.4. Periodicidade 67
8.5. Outras anotações 67
Frações parciais 68
Estatística (SME0123) 69
1. Probabilidade básica 70

2. Funções de probabilidade 72
3. Distribuições 74
3.1. Distribuições Discretas 74
3.2. Distribuições Contínuas 79
3.3. Notas adicionais 82
4. Variáveis Aleatórias Bidimensionais 84

CAPÍTULO

1

Arquitetura e Organização de Computadores

SCC0902

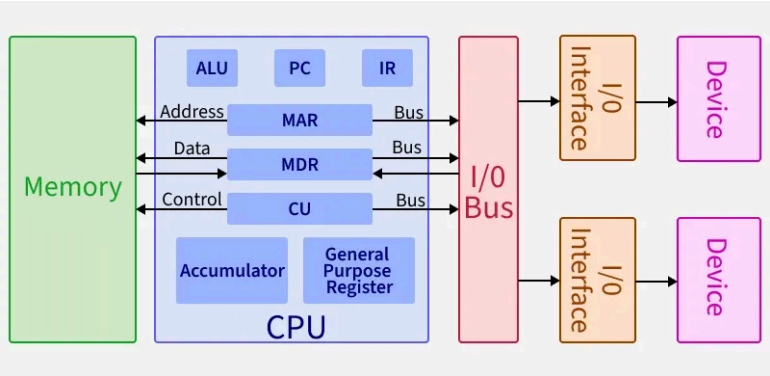
1. Primeira Parte

1.1. Arquitetura de Von-Neumann

Design de computador onde as instruções e dados são armazenados no mesmo espaço de memória. Os componentes são:

- CPU (CU + ALU)
- Registradores armazenados na CPU
- Dispositivos I/O operados pela CU
- Memória

CPU



- **CU (Control Unit)** - Gerencia como o processador funciona mandando sinais de controle (0/1) ao decodificar instruções.
- **ALU (Arithmetic and Logic Unit)** - Parte da CPU que lida com cálculos. Performa adição, subtração e operações lógicas, como comparações e deslocamento de bits em dados.

Registers

Tipo mais rápido de memória localizado na CPU.

- **PC (Program Counter)** - Endereço da próxima instrução a ser executada
- **IR (Instruction Register)** - Mantém a instrução atual sendo executada
- **MAR (Memory Address Register)** - Armazena o endereço da localização de memória sendo acessada
- **MDR (Memory Data Register)** - Armazena dados transferidos da/para a memória
- **Accumulator** - Armazena resultados intermediários de aritmética e operações lógicas
- **General Purpose Registers** - Uso para armazenamento temporário de dados

Bus

Sistema de comunicação que transfere dados, endereços, e sinais de controle entre a CPU, memória e dispositivos I/O. Na arquitetura de Von Neumann, um único bus é compartilhado para ambos dados e instruções.

1.2. Ciclo de instrução

Dois passos:

- Ciclo de busca (fetch)
- Ciclo de execução (execute)

Ciclo de busca

- PC tem endereço da próxima instrução

- Processador busca instrução na posição de memória armazenada no PC
  - MAR = PC
  - MBR = memória(MAR)
- PC = PC+4 (exceto em branch)
- Instrução armazenada no IR - IR = MBR

Ciclo de execução

- CU decodifica instrução e define sinais de controle
- Execução da instrução se resume a uma das seguintes possibilidades:
  - **Processador-memória**: Transferir dados do processador para a memória ou vice-versa
  - **Processador-I/O**: Transferir dados entre processador e dispositivo I/O
  - **Processamento de dados**: Executar operações aritméticas/lógicas sobre os dados
  - **Controle**: Especifica que a sequência de execução de instruções seja alterada
  - **Combinação dessas 4 possibilidades**

1.3. Arquitetura RISC-V

- Arquitetura de 32 bits
- 32 registradores de propósito geral
- 32 registradores para float

Compilando .asm

1. `sudo apt install nasm`
2. `sudo apt install binutils`
3. `nasm -f elf64 file.asm -o file.o` (monta arquivo)
4. `ld file.o -o file` (cria executável)

Estrutura de código

- `.data` - Início da seção de dados estáticos
- `.text` - Início da seção de código
- `.global label` - Entry point. Permite que outros arquivos chamem essa função durante o processo de link.

Diretivas

- `.align n` - Alinha a memória em 2^n bytes. Força o próximo dado/instrução a começar em um endereço de memória alinhado, utilizando padding.
  - `n = 0` para strings.
  - `n = 2` para instruções.
- `label: .asciz "str"` - Guarda string e encerra ela com `\0`
- `label: .byte b1,...,bn` - Reserva e inicializa cada byte sequencialmente.
- `label: .word w1,...,wn` - Reserva e inicializa cada word sequencialmente.
- `label: .double d1,...,dn` - Reserva e inicializa cada double sequencialmente.
- `label: .space n` - Reserva n bytes

Registradores

- `ra` - Return address
- `sp` - Stack pointer
- `t0-t6` - Temporary
- `a0-a6` - Function arguments
- `a7` - Syscall register
- `s0-s11` - Saved values

Comandos

Referência: Risc-V Cheat Sheet

Aritmética

- `add rd, rs1, rs2` - `rd = rs1 + rs2` (2 registradores)
- `addi rd, rs1, imm` - `rd = rs1 + imm` (1 registrador, 1 valor imediato/constante)
- `sub rd, rs1, rs2` - `rd = rs1 - rs2`
- `mul rd, rs1, rs2` - `rd = rs1 * rs2`
- `div rd, rs1, rs2` - `rd = rs1 / rs2`
- `rem rd, rs1, rs2` - `rd = rs1 % rs2`
- `remu rd, rs1, rs2` - `rd = rs1 % rs2` (unsigned remainder)
- `xor rd, rs1, rs2` - `rd = rs1 xor rs2`

Load/Store

- `la rd, label` - `rd = label_addr`
- `lw rd, imm(rs1)` - `rd = mem[rs1+imm]` (word = 4 bytes)
- `lb rd, imm(rs1)` - `rd = mem[rs1+imm]` (byte)
- `sw rs2, imm(rs1)` - `mem[rs1+imm] = rs2`
- `sb rs2, imm(rs1)` - `mem[rs1+imm] = rs2`

Jump/Function

- `j label` - `pc += label` (Obs: Usa half-words para pular. `jmp 4` -> `pc += 4 * 2 bytes`)
- `jal rd, label` - `rd = pc+4; pc += label` (salva endereço de retorno)
- `jr ra` - `pc = ra`
- `call symbol` - `ra = pc+4; pc = &symbol`
- `ret` - `pc = ra`

Branch (imm pode também ser um label)

- `beq rs1, rs2, imm` - `if(rs1 == rs2) pc += imm`
- `bne rs1, rs2, imm` - `if(rs1 != rs2) pc += imm`
- `blt rs1, rs2, imm` - `if(rs1 < rs2) pc += imm`
- `ble rs1, rs2, imm` - `if(rs1 <= rs2) pc += imm`
- `bgt rs1, rs2, imm` - `if(rs1 > rs2) pc += imm`
- `bge rs1, rs2, imm` - `if(rs1 >= rs2) pc += imm`
- `bnez rs1, imm` - `if(rs1 != 0) pc += imm`

Comparadores

- `slt rd, rs1, rs2` - `rd = (rs1 < rs2)`
- `slti rd, rs1, imm` - `rd = (rs1 < imm)`

Ecalls

- `a7` - Armazena número da ecall
- `ecall` - Chama a ecall

Chamadas ao sistema (ecall)

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints an floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Read an int from input console	N/A	a0 = the int
ReadFloat	6	Read a float from input console	N/A	fa0 = the float
ReadDouble	7	Reads a double from input console	N/A	fa0 = the double
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = maximum number of characters to read	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Read a character from input console	N/A	a0 = the character
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A

Alocando na stack

Empilhando

- `addi sp, sp, -8` - Reserva 8 words
- `sw a0, 4(sp)` - Salva `a0` na stack (sp+4)
- `sw ra, 0(sp)` - Salva `ra` na stack (sp)

Desempilhando

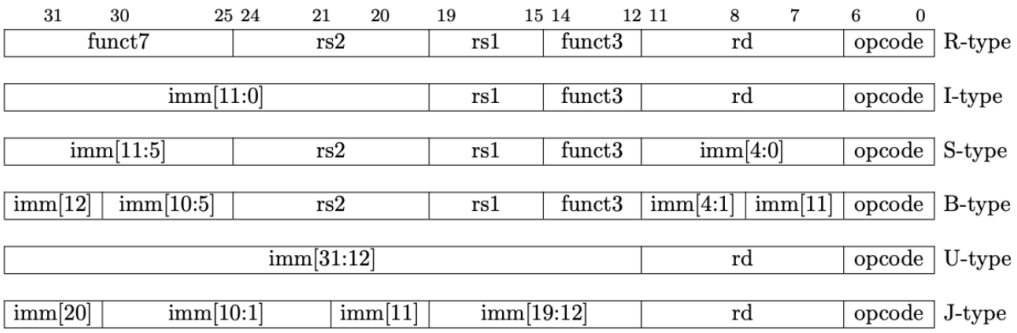
- `lw a0, 4(sp)`
- `lw ra, 0(sp)`
- `addi sp, sp, 8`

Instruction Set Architecture

Modelo abstrato que define a interface programável da CPU do computador, definindo como o software interage com o hardware. Um dispositivo que interpreta as instruções descritas por uma ISA é uma implementação da ISA.

- Quantidade e função dos registradores
- Formato das instruções (6 tipos)

Formato das instruções (6 tipos)



Tipo R (register)

Operações matemáticas e lógicas: `add`, `sub`, `and`, `or`, `sll`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem
- `funct7` - Auxílio para definição da operação

Ex: `add rd1, rs1, rs2` -> `0000000 01000 01001 000 10010 0110011`

Tipo I (immediate)

Operações aritméticas com constantes ou instruções de leitura da memória: `addi`, `lw`, `andi`, `jalr`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `imm` - Valor imediato

Ex: `lw s2, 0(sp)` -> `000000000000 00010 010 10010 0000011`

Tipo S (Store)

Escrever dados de um registrador para a memória: `sw`

- `opcode` - Código da operação
- `imm[4:0]` e `imm[11:5]` - Valor imediato (Separa os bits)
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `sw s2, 0(sp)` -> `00000000 10010 00010 010 00000 0100011`

Tipo B (branch)

Desvios condicionais: `beq`, `bne`, etc

- `opcode` - Código da operação
- `imm[4:0]`, `imm[11]`, `imm[11:5]`, `imm[12]` - Valor imediato
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `beq s1, s0, 4` -> `0 000000 01000 01001 000 0100 0 1100011`

Tipo U (Upper immediate)

Carregar números muito grandes diretamente nos bits superiores de um registrador: `lui`, `auipc`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[31:12]` - Valor imediato

Ex: `lui s0, 0x01234` -> `[0000 0001 0010 0011 0100] 01000 0110111`

Tipo J

Pulos incondicionais de longo alcance (chamadas de função e desvios obrigatórios): `jal`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[20]`, `imm[10:1]`, `imm[11]`, `imm[19:12]` - Valor imediato

Ex: `jal s0, -4` -> `1 1111111100 1 11111111 01000 1101111`

1.4. Monociclo (RISC-V)

A instrução é executada em um único ciclo de clock. Final de ciclo: tudo o que está nas conexões (linhas) é salvo nos registradores.

Componentes: PC, Banco registradores, ULA, UC, Memória, I/O.

Conjunto reduzido de instruções:

- Tipo R: `add`, `sub`, `and`, `or`
- Tipo I: `lw`
- Tipo S: `sw`
- Tipo B: `beq`

Metodologia de Clocking

Quaisquer valores armazenados em um elemento lógico sequencial são atualizados apenas em uma transição de clock: `State 1 -> Combinational Logic -> State 2`, em apenas 1 ciclo de clock.

Componentes

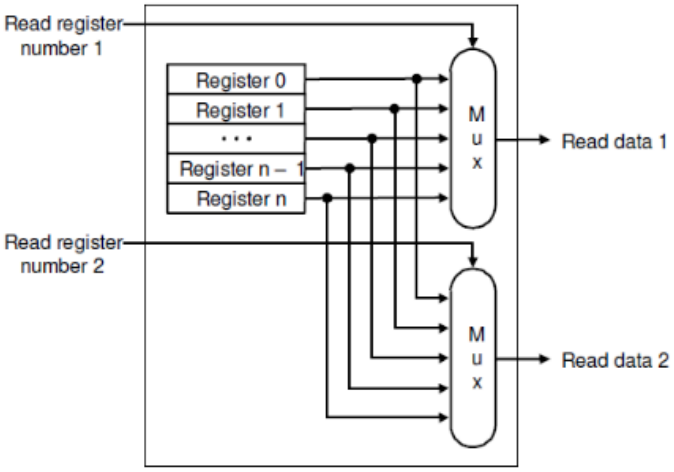
Arquitetura do banco de registradores

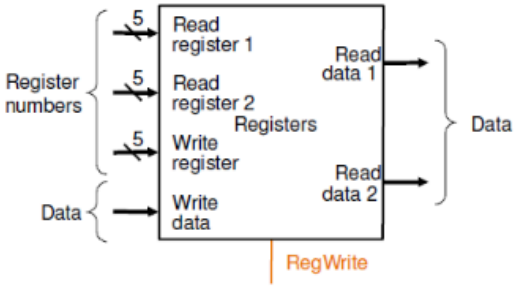
READ

1. 32 registradores
2. A saída de cada registrador vai para 2 MUX (saída duplicada)
3. 5 bits entram no Select do primeiro MUX -> Read Register 1 (5 bits)
4. 5 bits entram no Select do segundo MUX -> Read Register 2 (5 bits)
5. Saída do primeiro MUX -> Read Data 1 (32 bits)
6. Saída do segundo MUX -> Read Data 2 (32 bits)

WRITE

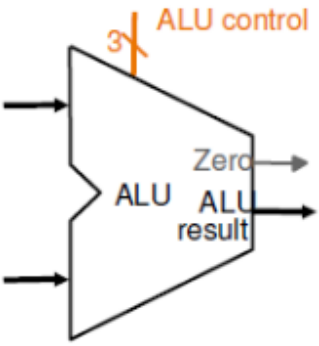
1. 32 registradores
2. Entrada Write Register (5 bits)
3. Entrada Write Data (32 bits)





Arquitetura da ALU

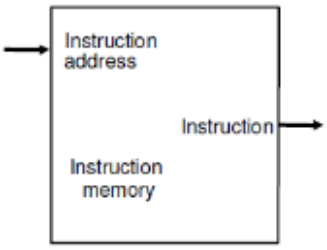
1. Recebe 2 entradas (32 bits)
2. Devolve saída (32 bits)



Arquitetura de operações com memória

Mem. Instrução:

1. Recebe endereço do PC (32 bits)
2. Devolve instrução (32 bits)

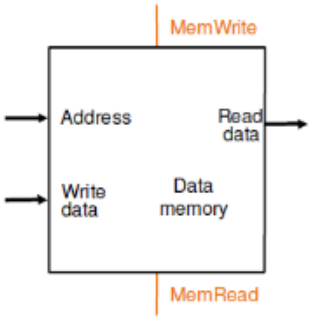


Mem. Dados:

1. Recebe endereço da ULA (32 bits) -> Devolve Read Data (32 bits)

ou

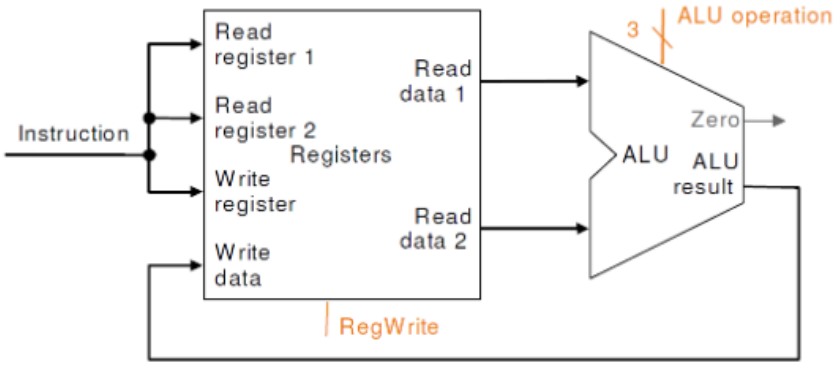
1. Recebe endereço (32 bits) -> Escreve Write Data (32 bits)



Execução de instruções

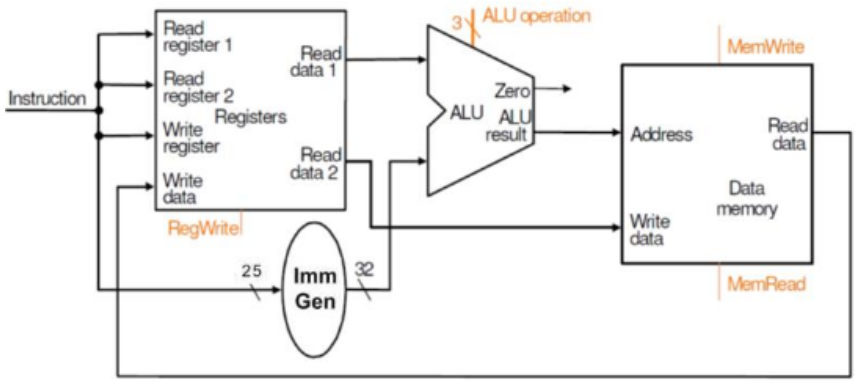
Execução das instruções tipo R

- Leitura de dois registradores
- Realização de operação aritmética/lógica
- Escrita do resultado no registrador



Execução de load/store

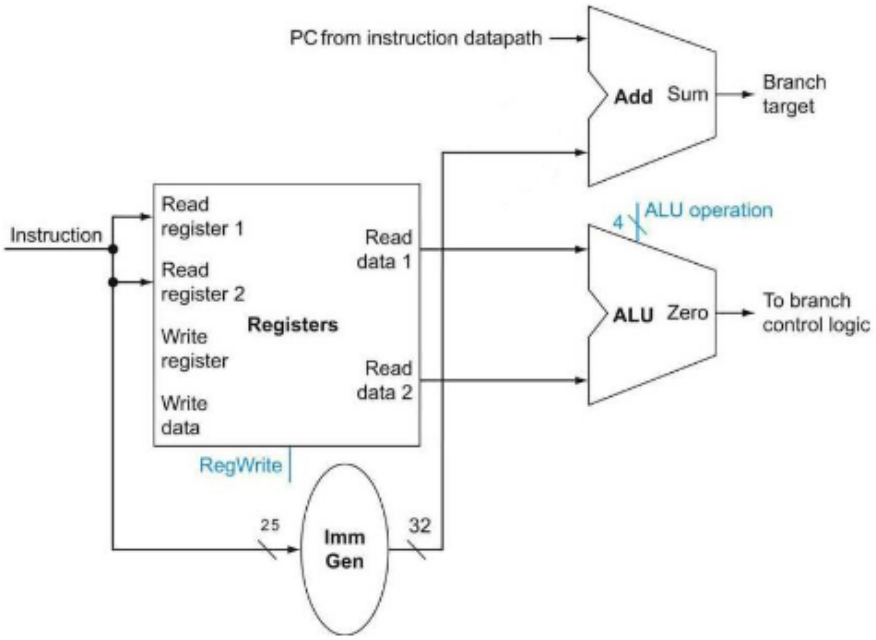
- **Load**: Leitura de registrador origem e valor imediato
  - Calcula endereço usando deslocamento de 12 bits (usa ALU)
  - Lê a memória e atualiza o registrador destino.
- **Store**: Leitura de dois registradores origem e valor imediato
  - Calcula endereço usando deslocamento de 12 bits (usa ALU)
  - Escreve o valor do registrador na memória.



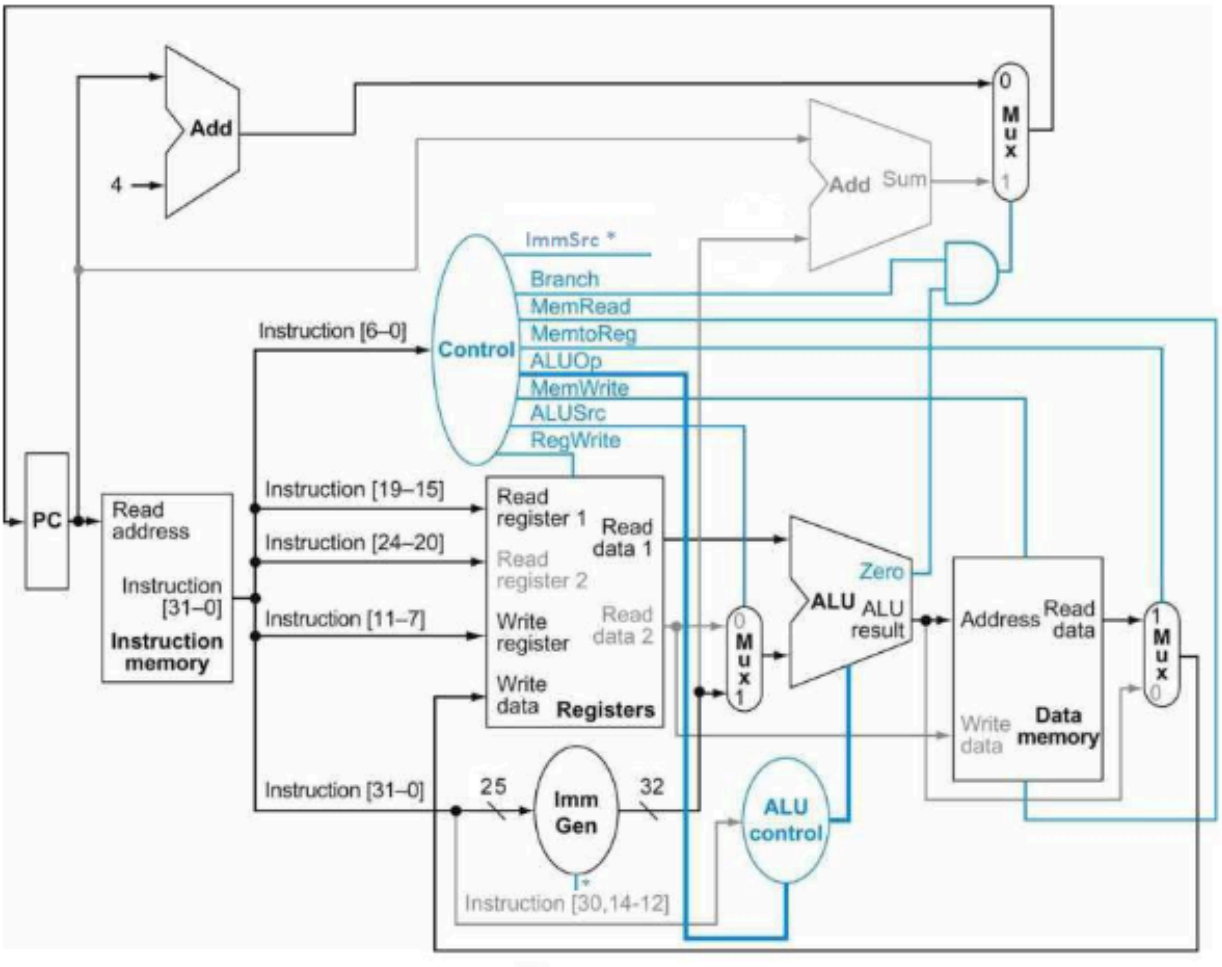
Execução de Branch

- Leitura de registradores

- Comparação dos dois através da ALU (se forem iguais, `zero = 1`)
- Endereço destino = PC + offset



Arquitetura completa

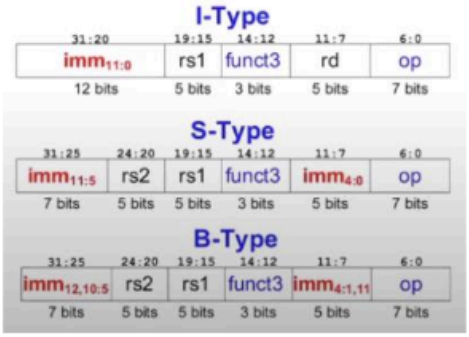


Tabelas e Blocos Auxiliares

ALU control lines	Function
000	AND
001	OR
010	add
110	subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	010
sw	00	store word	XXXXXXX	XXX	add	010
beq	01	branch if equal	XXXXXXX	XXX	subtract	110
R-type	10	add	0000000	000	add	010
R-type	10	sub	0100000	000	subtract	110
R-type	10	and	0000000	111	AND	000
R-type	10	or	0000000	110	OR	001

$ImmSrc_{1:0}$	ImmExt	Instruction Type
00	{{20{instr[31]}}, **instr[31:20]**}	I-Type
01	{{20{instr[31]}}, **instr[31:25], instr[11:7]**}	S-Type
10	{{19{instr[31]}}, **instr[31], instr[7], instr[30:25], instr[11:8], 1'b0**}	B-Type



Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0	ImmSrc
R-format	0	0	1	0	0	0	1	0	XX
lw	1	1	1	1	0	0	0	0	00
sw	1	X	0	0	1	0	0	0	01
beq	0	X	0	0	0	1	0	1	10

1.5. Notas avulsas

–4 em binário (20 bits)

Ao invés de usar bit de sinal, podemos usar complemento de dois: Inverte e soma 1.

```
00000000000000000100 -> 4
11111111111111111011 -> Complemento
+                  1
--------------------
11111111111111111100 -> -4
```

Formato de números inteiros

- 1 bit de sinal, 15 bits de magnitude.

2. Revisão de Lógica digital

2.1. Circuitos combinacionais e sequenciais

- Circuitos combinacionais - Função apenas de sinais de entrada.
- Circuitos sequenciais - Sinal de saída depende dos sinais anteriores de saída.

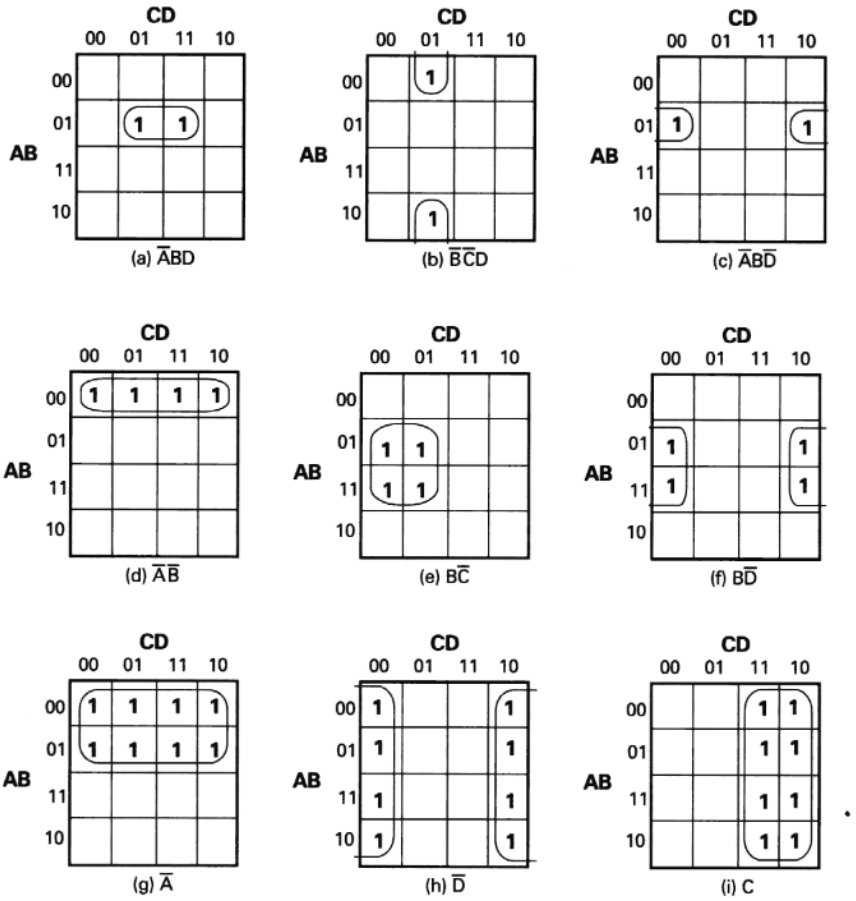
2.2. Identidades básicas

Basic Postulates		
$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative Laws
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive Laws
$1 \cdot A = A$	$0 + A = A$	Identity Elements
$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$	Inverse Elements
Other Identities		
$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Associative Laws
$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$	DeMorgan's Theorem

2.3. Mapas de Karnaugh

Bits em ordem de Gray code (apenas 1 bit muda de estado a cada incremento/decremento).

Soma de produtos: Agrupar 1s em clusters de 1, 2, 4, 8, ...



== Principais circuitos

- Half Adder - 2 entradas, 1 saída, 1 Cout.
- Full Adder - 2 entradas, 1 Cin, 1 saída, 1 Cout.
- Demux - Entrada de n bits, saída de 2^n bits. Somente uma saída é ativada para cada combinação de entradas.
- Mux - Até 2^n entradas e n valores seletores. Direciona uma das entradas para a saída.

2.4. Flip Flops

São circuitos sequenciais.

Flip-Flop S-R

- Duas entradas: S (set) e R (reset)
- Duas saídas: Q e $|Q|$
- $S = R = 1$ deve ser evitada. Resolve-se com flip-flop de entrada única (Flip-Flop D).

Flip-Flop J-K

- $J = K = 0$ - Mantém
- $J = 1, K = 0$ - Saída = 1
- $J = 0, K = 1$ - Saída = 0
- $J = K = 1$ - Inverte

2.5. Registradores

Exemplo de uso de flip-flops.

- Registradores paralelos - Carregamento e descarregamento feito simultaneamente.

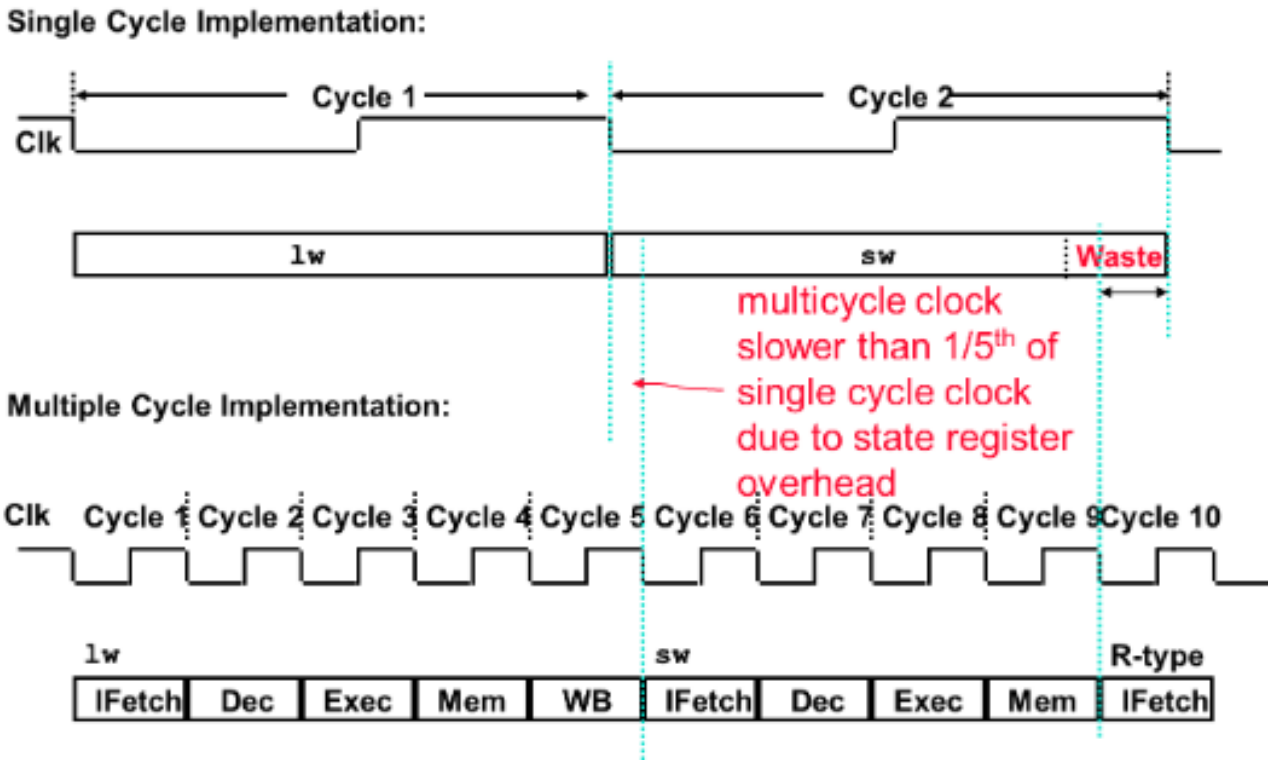
- Registradores de deslocamento - Transfere informações serialmente. Uso em I/O e também dentro da ALU (deslocamento lógico e rotação).
- Contadores - Registrador cujo valor é incrementado de 1 módulo a capacidade d

3. Segunda Parte

3.1. Multiciclo

A desvantagem do monociclo é que o uso do ciclo de clock se torna ineficiente, pois deve ser do tamanho da instrução mais lenta. Assim, instruções rápidas ficam aguardando tempo demais pelo fim do ciclo de clock.

No multiciclo, os ciclos de clock são menores, e cada ciclo de clock é ume estágio da instrução: IFech, Dec, Exec, Mem, WB. Nem todas as instruções demoram o mesmo número de ciclos de clock para finalizar.



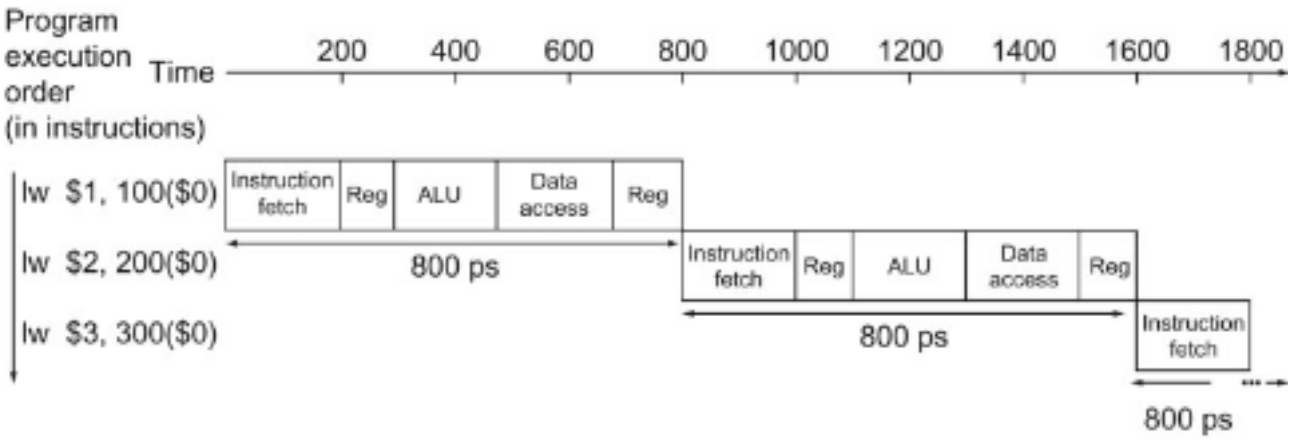
3.2. Pipeline

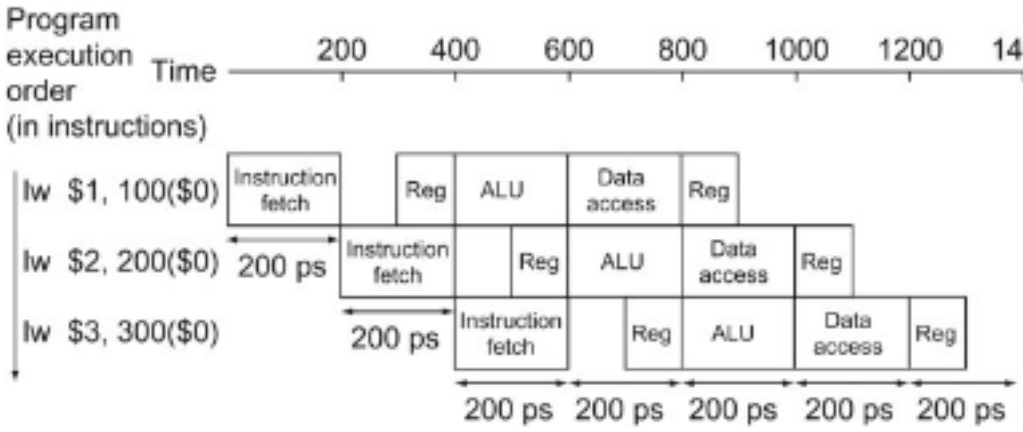
Dividir a instrução em etapas, cada etapa vai ser executada em 1 ciclo de clock, com sobreposição de etapas diferentes, de instruções diferentes, no mesmo ciclo de clock. Ideia de linha de montagem.

Etapas

- IF: Instruction Fetch - Mem. Instr.
- ID: Instruction Decode - UC/Banco Reg.
- EX: Execution - ULA
- Mem: Memory Access - Mem. Dados
- WB: Write Back - Banco de Registradores

Lembre-se: IF, ID, EX, MEM, WB





Comparação entre Monociclo, Multiciclo e Pipeline

- Monociclo: Cada instrução é executada completa em um único ciclo de clock. O ciclo precisa ser longo o suficiente para aguentar a instrução mais lenta.
- Multiciclo: Divide a execução de uma instrução em várias etapas menores. O ciclo é curto e baseado na etapa mais longa. instruções diferentes usam um número diferente de ciclos. Uma nova instrução só começa quando a anterior termina.
- Pipeline: Divida a instrução em estágios, mas permite sobreposição.

Para 100 instruções:

Obs: IPC = instruction per clock, CPI = clock per instruction

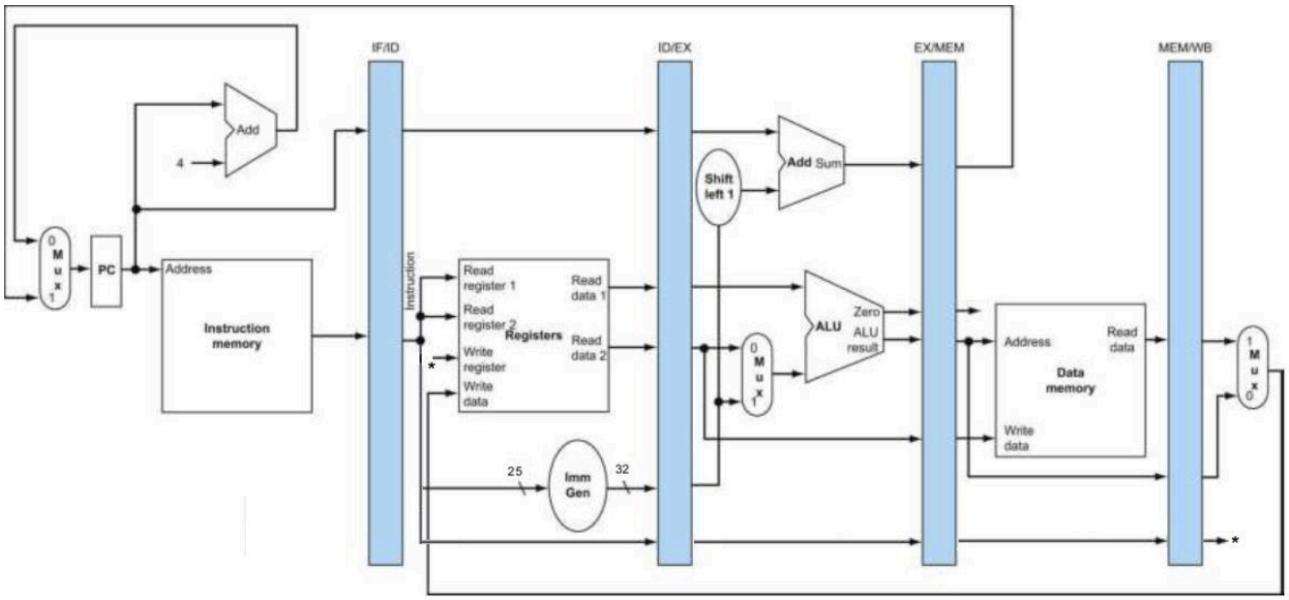
- Monociclo:
  - Tempo de clock = 45 ns
  - Para 100 instruções: $45 \cdot 100 = 4500$ ns
- Multiciclo:
  - Tempo de clock = 10 ns
  - CPI médio = 4.2 clocks/inst.
  - Para 100 instruções: $4.2 \cdot 10 \cdot 100 = 4200$ ns
- Pipeline:
  - Tempo de ciclo de clock = 10 ns
  - N° de ciclos de clock = (1 inst: 5, 2 inst: 6, 3 inst: 7). Isso pois precisa primeiro "encher" o pipeline.
  - Para 100 instruções: 104 ciclos de clock - $104 \cdot 10 = 1040$ ns
  - Problema: paradas (gargalos - Instruções esperando liberar um componente, disputa por recursos)

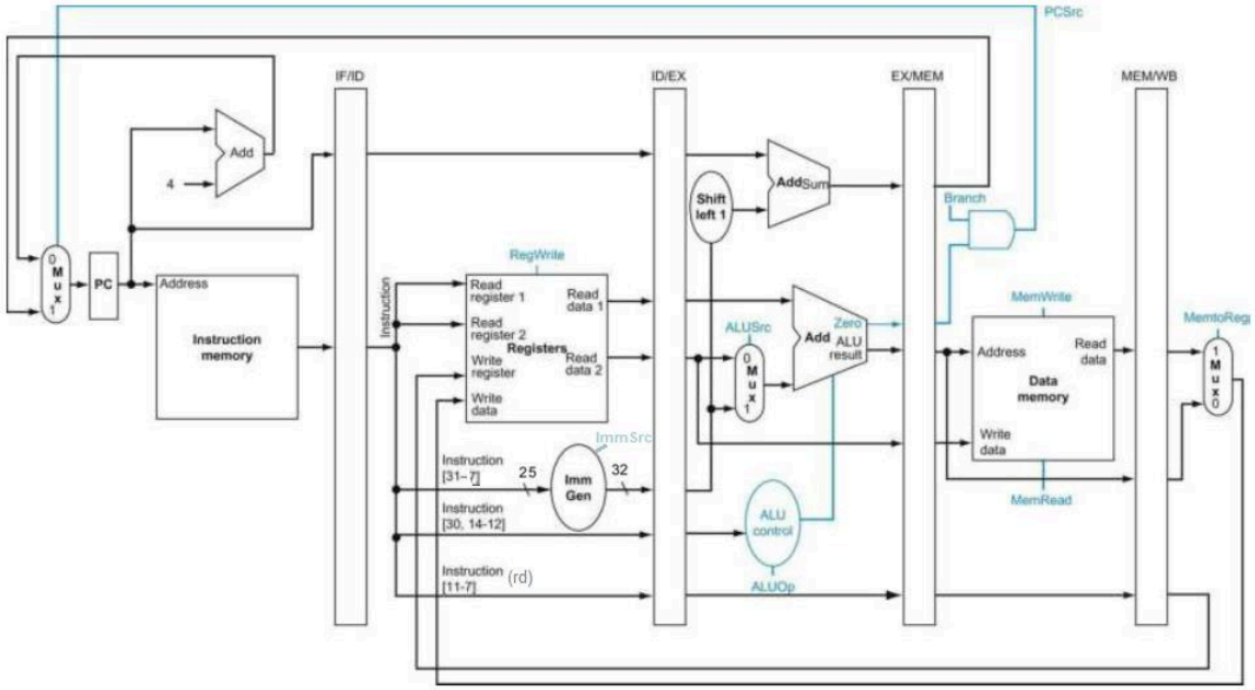
o registrador.

Considerações

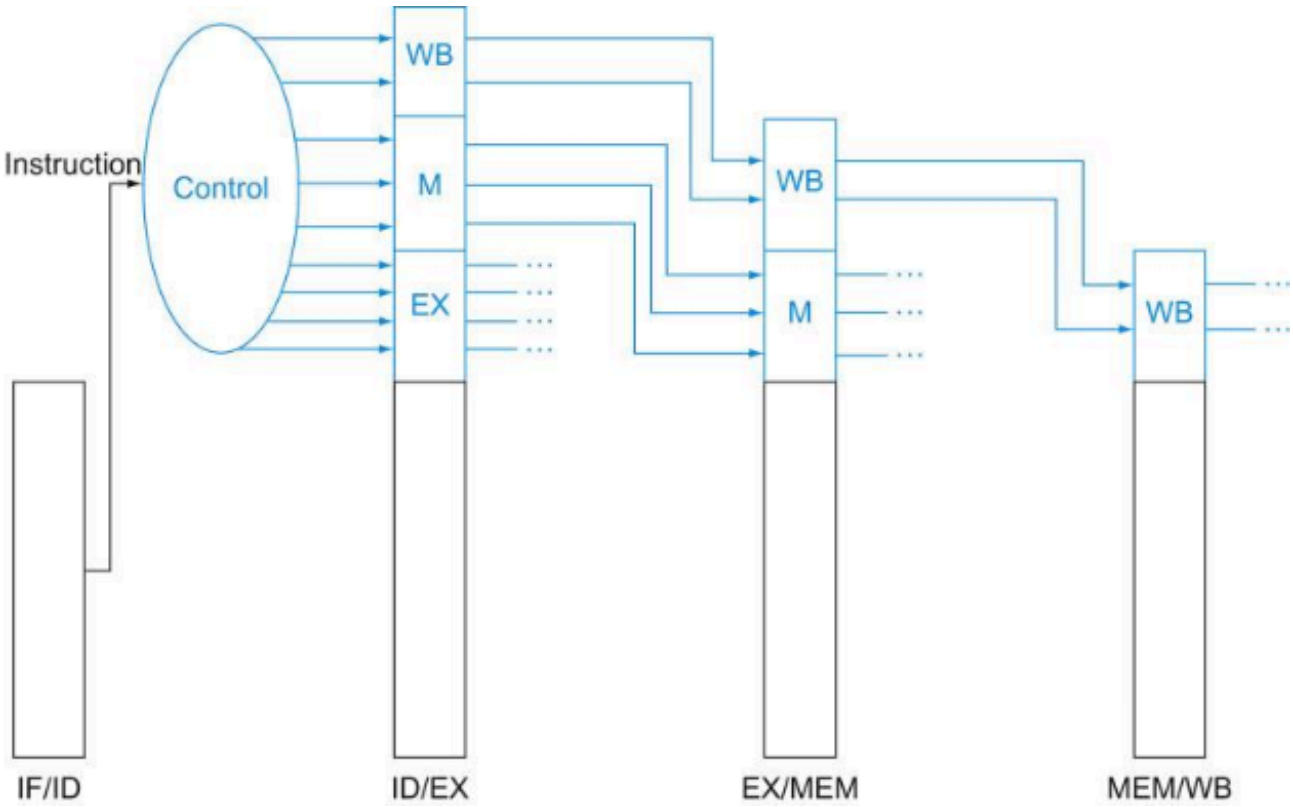
- O pipeline não diminui o tempo de execução de cada tarefa, mas reaproveita módulos não utilizados (aumenta throughput)
- A taxa do pipeline é limitada pelo estágio mais lento
- O pipeline trava se houver dependências

Registradores intermediários do pipeline





Sinais de controle:



Dependências ou conflitos

Conflitos estruturais

Acessos simultâneos à memória feitos por 2+ estágios.

- Acessos concorrentes à memória - Uso de memórias multi-portas ou com múltiplos bancos com acessos independentes
  - Soluções: Caches separadas de dados e instruções, memória com múltiplos de acesso independentes
- Leitura de instrução e leitura/escrita de dados simultâneos à memória - Uso de arquitetura "Harvard" com caches de dados e instrução separados
- Acesso simultâneo ao banco de registradores - Uso de banco de registradores com múltiplas portas
- Uso simultâneo de uma mesma unidade funcional - Replicação da unidade funcional ou implementação "pipelined" dessa unidade

Dependência de dados

Uma instrução faz uso de um operando que vai ser produzido por uma outra instrução que ainda está no pipeline. A execução da instrução seguinte depende do operando calculado pela instrução anterior.

Tipos de dependência de dados:

Dependência verdadeira (diretas) ou RAW (Read After Write) - Uma instrução utiliza um operando que é produzido por uma instrução anterior

```riscv
add <R1>, R2, R3
sub R4, <R1>, R6
```

Dependência falsa - Antidependência ou WAR (Write After Read) - Uma instrução lê um operando que é escrito por uma instrução sucessora

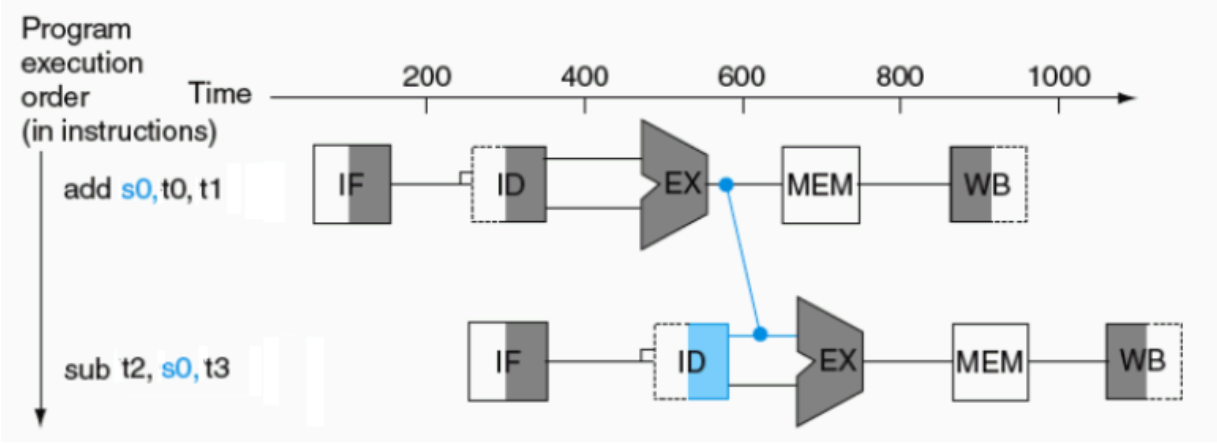
```riscv
sub R4, <R1>, R6
add <R1>, R2, R3
```

Dependência de saída ou WAW (Write After Write) - Uma instrução escreve em um operando que é também escrito por uma instrução sucessora

```riscv
add <R4>, R2, R3
sub <R4>, R1, R6
```

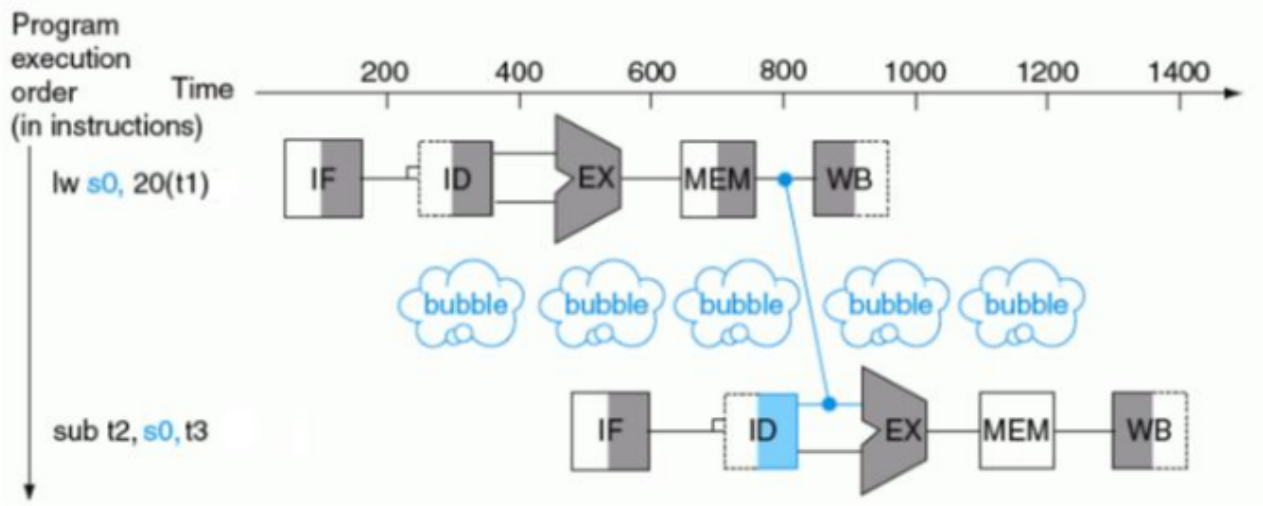
Adiantamento de Dados (forwarding)

Caminho interno dentro do pipeline entre a saída e a entrada da ULA. Técnica conhecida como forwarding ou bypassing. Evita a parada do pipeline utilizando **buffers internos** em vez de esperar que o elemento de dado chegue nos registradores visíveis ao programador ou na memória.



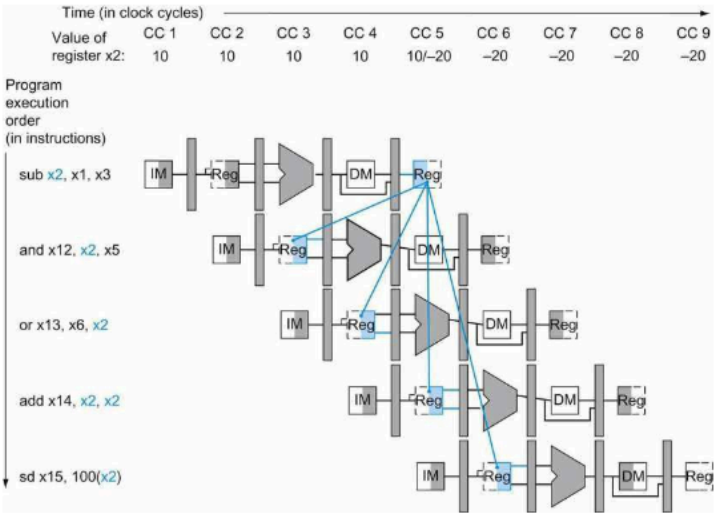
Perceba que adiantamos a subtração de `s0` após ele ficar livre da primeira instrução.

Mas, em algumas situações, nem o forwarding pode resolver o problema de parada do pipeline

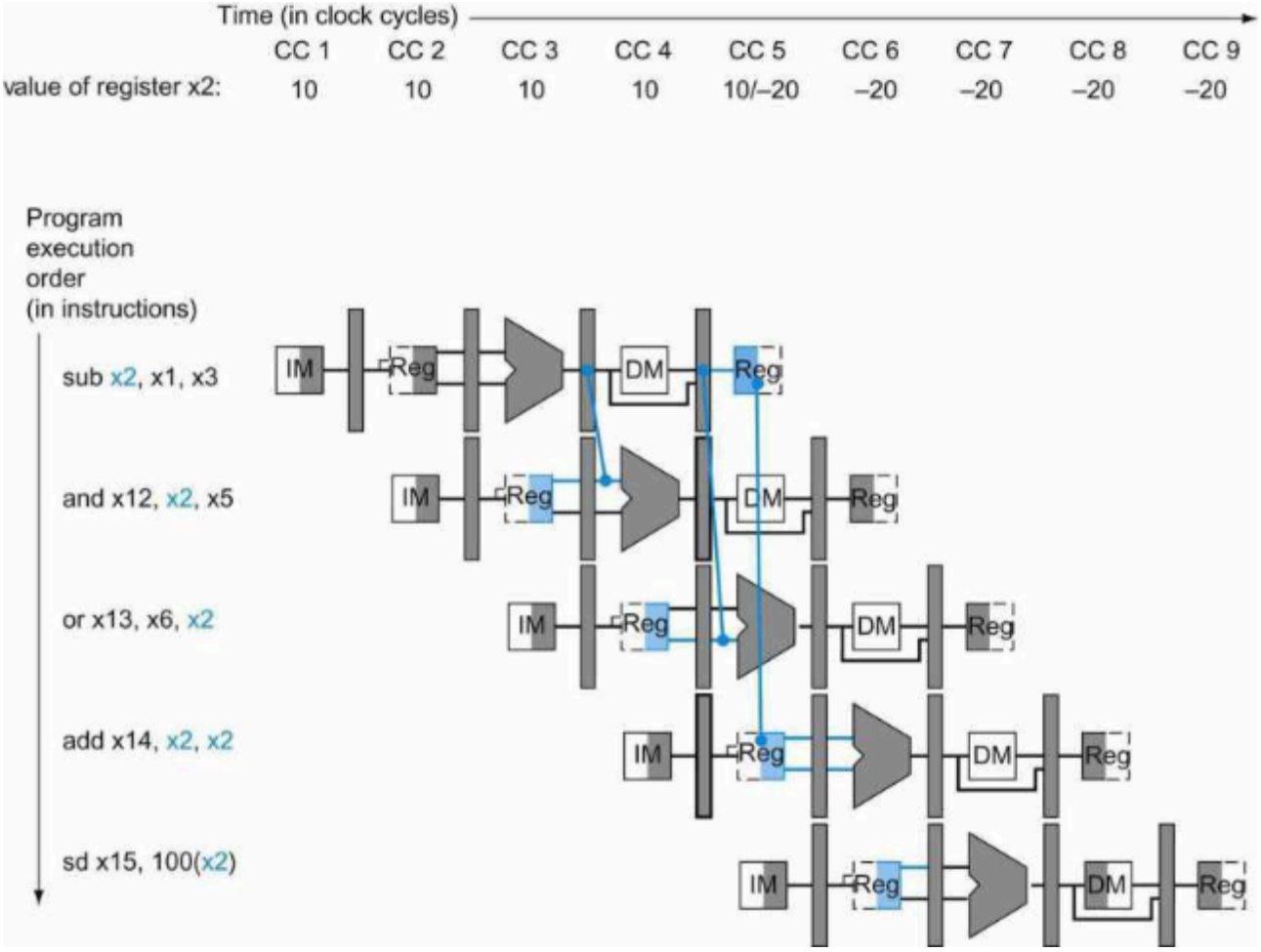


Se as próximas instruções utilizam o valor do registrador, mas não escrevem nele, dá pra replicar ele para todas as próximas instruções. Observe sem forwarding:

```
sub x2, x1, x3 // Register z2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sd x15, 100(x2) // Base (x2) depends on sub
```

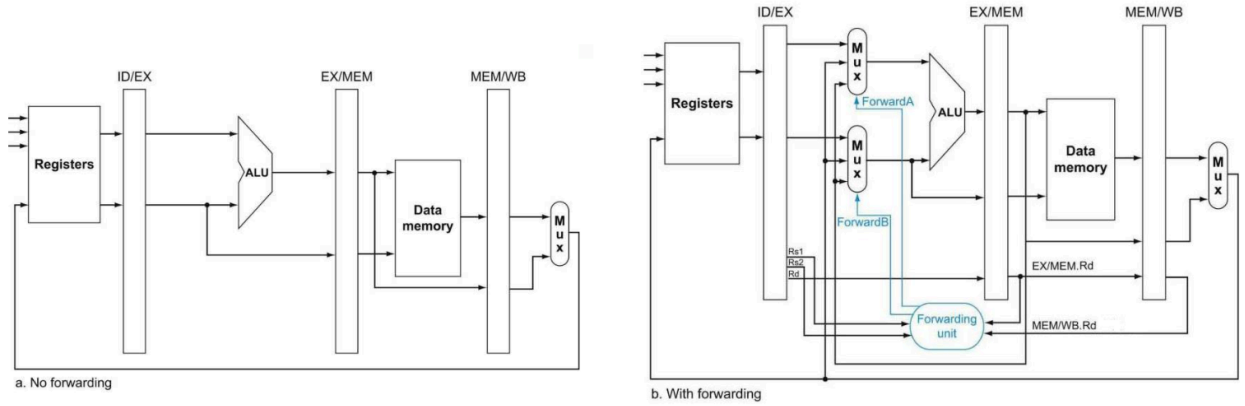


E agora com forwarding:



Condições para que o adiantamento seja considerado:

- Destino da instrução que está entre os ciclos de **EX e MEM** é igual a um dos registradores de origem da instrução que está entre os ciclos de ID e EX
- Destino da instrução que está entre os ciclos de **MEM e WB** é igual a um dos registradores de origem da instrução que está entre os ciclos de ID e EX



Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Condições para detecção de dependência de dados:

- Dependência EX:

```riscv
if (EX/MEM.RegWrite and (EX/MEM.Rd != 0) and (EX/MEM.RegWrite = ID/EX.Rs1)) ForwardA = 10

if (EX/MEM.RegWrite and (EX/MEM.Rd != 0) and (EX/MEM.Rd = ID/EX.Rs2)) ForwardB = 10
```

- Dependência MEM:

```riscv
if(MEM/WB.RegWrite and (MEM/WB.Rd != 0) and (MEM/WB.Rd = ID/EX.Rs1)) ForwardA = 01

if(MEM/WB.RegWrite and (MEM/WB.Rd != 0) and (MEM/WB.Rd = ID/EX.Rs2)) ForwardA = 01
```

Problema: Quando se está acumulando os valores de um vetor em um registrador e lê e escreve no mesmo registrador.

```riscv
add 1,1,2
add 1,1,3
add 1,1,4
...
```

Agora mudamos para a dependência MEM.

```riscv
if(MEM/WB.RegWrite and (MEM/WB.Rd != 0) and(EX/MEM.rd != ID/EX.rs1) and (MEM/WB.Rd = ID/EX.Rs1))
ForwardA = 01

if(MEM/WB.RegWrite and (MEM/WB.Rd != 0) and (EX/MEM.rd != ID/EX.rs2) and (MEM/WB.Rd = ID/EX.Rs2))
ForwardA = 01
```

CAPÍTULO

2

Matemática Discreta

SMA0180

1. Contagem

Conceitos importantes

- *Disjunto* - Dois conjuntos são disjuntos se não possuem elemento em comum.
- *Mutuamente disjuntos* - Um conjunto de conjuntos é uma família de conjuntos mutuamente disjuntos se todos os pares possíveis S_i são disjuntos.
- *Tamanho de um conjunto* - Número de elementos distintos em S

Princípio da soma

O tamanho de uma união de conjuntos finitos **mutuamente disjuntos** é a soma do tamanho dos conjuntos.

$$|\cup_{i=1}^{m} S_i| = \sum_{i=1}^{m} |S_i|$$

Example.

```
i = 1, ..., r
  j = 1, ..., m
    k = 1, ..., n
```

$$|S_{i,j}| = n$$

$$|T_i| = |\cup_{j=1}^{m} S(i,j)| = \sum_{j=1}^{m} S(i,j) = \sum_{j=1}^{m} n = m \cdot n$$

$$\text{Total} = |\cup_{i=1}^{r} T_i| = \sum_{i=1}^{r} T_i = \sum_{i=1}^{r} m \cdot n = m \cdot n \cdot r$$

□

Princípio do produto

O tamanho de uma união de m conjuntos disjuntos de tamanho n é igual a $m \cdot n$.

Ou então, se temos *n maneiras de realizar A* e *m maneiras de realizar B*, então temos $n \cdot m$ maneiras de realizar A e B.

2. Contagem de listas, permutações e subconjuntos

Uso do princípio da soma e do produto

Uma senha pode conter 4,5,6,7 ou 8 caracteres. Quantas senhas no total?

$$P = P_4 \cup P_5 \cup P_6 \cup P_7 \cup P_8$$

Como P_i são mutuamente disjuntos, pode-se aplicar o princípio da soma:

$$|P| = \sum_{i=4}^{8} |P_i|$$

Para cada letra, há 52 escolhas. Assim, pelo princípio do produto:

$$|P| = 52^4 + 52^5 + 52^6 + 52^7 + 52^8$$

Princípio da bijeção

Dois conjuntos possuem o mesmo tamanho se, e somente se, há uma função um-para-um de um conjunto em outro.

Permutação de k-elementos de um conjunto

Quantas permutações de $\{1, 2, \ldots, n\}$ podemos fazer? Ex: Escolher $k = 3$ pessoas diferentes de um grupo de n pessoas.

Lógica: $n \cdot (n-1) \cdot (n-2) \ldots$

$$\frac{n!}{(n-k)!}$$

Contando subconjuntos de um conjunto

Quantos subconjuntos **únicos** de k elementos pode-se fazer em um conjunto de tamanho n? A ordem não importa, ou seja, $\{1,2\} = \{2,1\}$. Deve-se remover as permutações. Usa-se coeficiente binomial para "escolher k dentre n":

$$\binom{n}{k} = \frac{n!}{(n-k)!k!}$$

Example.

Contar caminhos em 10 passos. Direções possíveis: $\{D, D, D, D, D, D, C, C, C, C\}$

- - - - - - - - - -

- $\binom{10}{4}$: Escolher 4 posições de 10 para C. Elimina repetições (CDC = CDC)
- Ou $\binom{10}{6}$: Escolher 6 posições de 10 para D.

□

3. Coeficientes binomiais

Relação de Pascal

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Teorema binomial

$$(x+y)^n = \sum_{i=0}^{n} \binom{n}{i} x^{n-i} y^i$$

Prova

$$(x+y)^3 =$$

$$= (xx + xy + yx + yy)(x+y) = (xx + xy + yx + yy)x + (xx + xy + yx + yy)y =$$

$$= xxx + xyx + yxx + yxx + xxy + xyy + yxy + yyy$$

- Coeficiente de $xxx = \binom{3}{0}$ (escolher 0 posições para y)
- Coeficiente de $xxy = \binom{3}{1}$ (escolher 1 posição para y)
- Coeficiente de $xyy = \binom{3}{2}$ (escolher 2 posições para y)
- Coeficiente de $yyy = \binom{3}{3}$ (escolher 3 posições para y)

4. Relações de equivalência e contagem

Relação entre conjuntos

Uma relação entre 2 conjuntos X, Y é um conjunto de pares ordenados:

$$R = \{(x, y) |\ x \in X, y \in Y \text{ satisfazem } ...\}$$

Quando $(x, y) \in R$, escrevemos $x \sim y$.

Propriedades:

1. Reflexiva - $x \sim x$
2. Simétrica - $x \sim y \Leftrightarrow y \sim x$
3. Transitiva - $x \sim y, y \sim z \Rightarrow x \sim z$

Uma relação é de equivalência se é reflexiva, simétrica e transitiva.

Dada uma relação de equivalência, as classes de equivalência são os conjuntos formados por elementos **relacionados** ($x = 1 \rightarrow \{y_1, y_2, ...\}$). De modo geral:

$$[x] = \{y | y \sim x\}$$

As classes de equivalência codificam simetrias/invariâncias.

5. Aritmética modular

Conjunto modular

$\mathbb{Z}_n$ = Inteiros módulo n. Possui as operações de adição e multiplicação.

Propriedades da adição modular

1. $$a \bmod n = (a + k \cdot n) \bmod n$$
2. $$(a + b) \bmod n = a \bmod n + b \bmod n$$

Adicionar/subtrair dos dois lados sempre conserva a igualdade.

Propriedades da multiplicação modular

1. $$(a \cdot b) \bmod n = (a \bmod n \cdot b \bmod n) \bmod n$$
2. $$a = b \pmod n \Rightarrow ac = bc \pmod n$$
3. $$a = b \pmod n \Rightarrow \frac{a}{c} = \frac{b}{c} \pmod n \Leftrightarrow mdc(c, n) = 1$$

Multiplicar dos dois lados sempre conserva a igualdade.

Dividir dos dois lados só conserva a igualdade se o divisor for coprimo com n (sem fatores em comum).

Coprimo

Um número a é coprimo com n se $mdc(a, n) = 1$. Isso também pode ser visto como a existência de inverso multiplicativo.

Inverso multiplicativo e Algoritmo estendido de Euclides

Dado $a \in \mathbb{Z}_n$, o inverso multiplicativo na operação de módulo é caracterizado por:

$$aa^{-1} = 1 \bmod n$$

Essa equação só tem solução se $mdc(a, n) = 1$ (Teorema de Bezault é válido para 1).

Algoritmo de Euclides

Dado $a > b$:

$$mdc(a, b)$$

$$a = k \cdot b + r$$

$$mdc(a, b) = mdc(r, b)$$

E assim por diante, até

$$mdc(x, 1)$$

.

Resumo: Pega maior e substitui pelo módulo com o menor.

Teorema de Bezault

Dados $a, n \in \mathbb{Z}$, $\exists \alpha, \beta \in \mathbb{Z}$ tal que:

$$mdc(a, n) = \alpha a + \beta n$$

Algoritmo estendido de Euclides

Dado o teorema de Bezault, o algoritmo estendido de Euclides consiste em encontrar α, β tal que:

$$\alpha \cdot a + \beta \cdot b = mdc(a, b)$$

Através do seguinte algoritmo:

1. Calcular mdc através do algoritmo de Euclides, guardando todas as expressões do tipo $a = k \cdot b + r$
2. Ao chegar em $r = mdc(a, b)$, deve-se colocar $mdc(a, b)$ em função de a, b, substituindo todos os restos, da última para a primeira expressão.

Example.

Dado $mdc(1097, 520) = 1$, queremos encontrar α, β tal que $\alpha \cdot 1097 + \beta \cdot 520 = 1$.

Primeiro, o algoritmo de Euclides.

$$mdc(1097, 520) = mdc(57, 520) = mdc(57, 7) = mdc(1, 7)$$

$$\begin{cases} 107 = 2 \cdot 520 + 57 \end{cases}$$

Teorema de Euler

Seja $m = \varphi(n)$. O conjunto dos **coprimos** (não todos os elementos, mas apenas os que possuem inverso multiplicativo!) de n é:

$$\{x_1, ..., x_m\}$$

Multiplicamos por a com $mdc(a, n) = 1$. Isso garante a inexistência de fatores em comum, e o novo conjunto contém os mesmos números do conjunto original, mas em uma ordem diferente. Nenhum se repete.

$$\{ax_1, ..., ax_m\}$$

Como os conjuntos contém os mesmos elementos, o produto de todos os elementos de cada conjunto é equivalente.

$$(x_1 \cdot ... \cdot x_m) = a^m \cdot (x_1 \cdot ... \cdot x_m) \bmod n$$

Como o produto $(x_1 \cdot ... \cdot x_m)$ é coprimo com n, podemos cancelá-lo dos dois lados da igualdade (**coprimo = $mdc(a, n) = 1$, logo para cada x_i existe um inverso multiplicativo**). O que sobra é o teorema de Euler.

$$(x_1 \cdot ... \cdot x_m)(x_1^{-1} \cdot ... \cdot x_m^{-1}) = a^m \cdot (x_1 \cdot ... \cdot x_m)(x_1^{-1} \cdot ... \cdot x_m^{-1}) \bmod n$$

$$\therefore 1 = a^{\varphi(n)} \bmod n$$

Isso significa que **as potências de um tamanho qualquer $a \in \mathbb{N}^*$ em módulo n posusi tamanho de ciclo que divide ou é igual a $\varphi(n)$**. A cada $\frac{\varphi(n)}{n}$ multiplicações, o resultado volta para 1.

Para encontrar o tamanho do ciclo, basta **encontrar o menor expoente positivo k que resulta em 1** para $a^k \bmod 7$ (ao chegar em 1, é confirmado o reset do ciclo). Se $k = 2$, os ciclos possuem tamanho 2.

Função totiente

Para n fatorado em primos de modo que $n = p_1^{a_1} \cdot p_2^{a_2} ...$, a função totiente é definida como:

$$\varphi(n) = n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) ...$$

E fornece a quantidade de números menores ou igual a n co-primos com respeito a ele (Ex: $\varphi(8) = 4$, pois 1,3,5,7 são co-primos). Isso fornece o **tamanho exato do conjunto de elementos que possuem inverso multiplicativo módulo n** (a tal que $mdc(a, n) = 1$), ou coprimos.

1) Em particular: dado $n = p \cdot q$, com p, q primos, a função totiente é calculada como:

$$\varphi(n) = (p - 1)(q - 1)$$

2) Em particular: dado n primo, $\varphi(n) = n - 1$.

Pequeno teorema de Fermat

Caso particular do teorema de Euler para p primo:

$$a^{p-1} = 1 \bmod p$$

Em particular, se $a \in \mathbb{Z}_p$ então:

$$a^1 \cdot a^{p-2} = 1 \bmod p$$

$$\therefore a^{-1} = a^{p-2} \bmod p$$

Na prática, isso permite simplificar potências gigantescas:

$$a^k = a^{k \bmod \varphi(n)} \bmod p$$

Example.

$$7^{1020} \bmod 103$$

$$\varphi(103) = 102$$

$$1020 = 10 \cdot (102) + 0$$

$$7^{1020 \bmod 102} \cdot 7^0 \bmod 103 = 7^{10 \cdot 102 \bmod 102} \cdot 7^0 \bmod 103 = 1 \bmod 103$$

□

Propriedades da potenciação modular

Definimos:

$$a^k \bmod n$$

Propriedades:

- $a^{k_1+k_2} \bmod n = a^{k_1} \cdot a^{k_2} \bmod n$
- Pequeno teorema de Fermat: $a^{\varphi(n)} \bmod n = 1 \bmod n$
- Pequeno teorema de Fermat: para a, n coprimos, $a^{-1} = a^{n-2} \bmod n$
- O tamanho de ciclo é um divisor de $\varphi(n)$.

Propriedade de ciclo para a, n coprimos:

Para encontrar o tamanho de ciclo K, basta encontrar K tal que:

$$a^K = 1 \bmod n_1$$

Propriedade de ciclo para a, n não coprimos:

A parte coprima n_1 é a multiplicação de todos os fatores de n que não compartilham primos com a.

A parte não coprima n_2 é a multiplicação de todos os fatores de n que compartilham primos com a.

Eles são tais que:

$$n = n_1 \cdot n_2$$

$$mdc(n_1, n_2) = 1$$

A parte coprima n_1 define o tamanho de ciclo K. Basta encontrar K tal que:

$$a^K = 1 \bmod n_1$$

A parte não coprima n_2 define um pré-período de modo que ele termina em $k = K - 1$. Bata encontrar K tal que:

$$a^K = 0 \bmod n_2$$

Example.

$$4^k \bmod 20$$

1. $10 = 2^2 \cdot 5$
2. $4 = 2^2$
3. $n_1 = 5$
4. $n_2 = 2^2$

Verificando:

- $20 = 5 \cdot 2^2$
- $mdc(5, 4) = 1$

Encontrando tamanho do ciclo:
Potenciação completa:

Achar solução de sistema de equações modulares

$$\begin{cases} x \bmod 5 = 4 \\ x \bmod 7 = 5 \end{cases}$$

1. Escolha uma equação e coloque na forma completa: $x = 5k + 4$
2. Substitua no valor de x na segunda equação
3. Ache o inverso modular do número que multiplica k e multiplique dos dois lados para obter k
4. Ao substituir k na equação inicial, temos x.

Obs: Na equação $5k = 2 (\bmod 7)$, basta encontrar $5^{-1} \bmod n$ e multiplicar dos dois lados. (isso que é "dividir" dos dois lados em aritmética modular).

Obs: Se a constante que multiplica k não tiver inverso multiplicativo, o sistema não possui solução.

Teorema Chinês do Resto

Dado um sistema de equações do tipo:

$$\begin{cases} x \bmod m_1 = a_1 \\ x \bmod m_2 = a_2 \\ \dots \\ x \bmod m_k = a_k \end{cases}$$

Se os módulos $m_1, m_2, ..., m_k$ forem primos entre si dois a dois (mdc = 1), o teorema garante que:

1. O sistema sempre possui uma **solução única** dentro de um período (x = A + kB).
2. A solução geral é dada módulo $M = m_1 \cdot m_2 ... m_k$ (produto dos módulos).

Caso os módulos não sejam primos entre si, ainda assim podem ter solução. As congruências precisam ser analisadas dois a dois, de modo que:

$$(a_1 - a_2) = k \cdot mdc(m_1, m_2), k \in \mathbb{Z}$$

Se a condição for verdadeira, o sistema possui solução, e ela é única módulo o mmc entre os módulos.

Se a condição for falsa, o sistema não possui solução inteira.

Teorema de Wilson

O teorema de Wilson afirma que:

$$(p-1)! \bmod p = (p-1)$$

Isso pois $(p-1)!$ é a multiplicação de todos os coprimos de p. Como todos são coprimos, todos possuem inverso multiplicativo.

Assim, aos pares, a multiplicação entre os coprimos resulta em 1. O inverso multiplicativo de $(p-1)$ é ele mesmo, de modo que ele é o termo que sobra.

Sempre serão formados pares formando apenas $(p-1)$, pois todos os primos são ímpares, exceto 2.

$$1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \bmod 7$$

- $1^{-1} = 1$
- $2^{-1} = 4$
- $3^{-1} = 5$
- $6^{-1} = 6$

Portanto:

$$[1] \cdot [2 \cdot 4] \cdot [3 \cdot 5] \cdot 6 \bmod 7 = 1 \cdot 6 \bmod 7$$

5.1. Anotações

Achar x de equação linear

Para calcular equações do tipo:

$$a \cdot_n x = b$$

Precisamos do inverso multiplicativo, de modo que:

$$x = b \cdot_n a^{-1}$$

Lembrando que o inverso multiplicativo só existirá se $mdc(a, n) = 1$, e é calculado por Euclides estendido.

multiplicando pelo inverso dos dois lados

Se p é primo e $a < p$, a sempre tem inverso multiplicativo:

$$ax \underset{p}{=} 1 \Leftrightarrow x \underset{p}{=} a^{-1}$$

Isso pois $mdc(p, a) = 1$ sempre (condição de existência de inverso multiplicativo).

Observações sobre multiplicação com a,n primos entre si

Para equação linear, com a, n primos entre si:

$$k \cdot a \bmod n$$

Como n é primo, $\varphi(n) = n - 1$. Essa é quantidade de elementos em um ciclo.

Na aritmética modular, quando você multiplica uma **sequência completa de restos** (k) por um **número que não compartilha fatores com o módulo** (a), o conjunto resultado não possui repetições, formando uma permutação perfeita. Essa demonstração é feita no teorema de Euler.

Para ilustrar, tome $n = 11$. Como $\varphi(n) = 10$, $k \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ (todos os restos possíveis). Tome $a = 5$. Pela mesma demonstração do teorema de Euler, **multiplicar o conjunto dos restos de mod n por uma constante qualquer mantém os mesmos restos, mas permutando sua ordem no conjunto** (por causa da existência de inverso multiplicativo). Sem repetição.

Obs: k, n e a, n devem ser primos entre si para isso ser válido (atesta existência de inverso multiplicativo).

Observações sobre multiplicação com a,n possuindo fatores em comum

Para equação linear, com a, n não primos entre si:

$$k \cdot a \bmod n$$

Definindo $d = mdc(a, n)$:

- Apenas múltiplos de d aparecerão na sequência do ciclo.
- O conjunto terá apenas $\frac{n}{d}$ números diferentes.
- Cada um desses valores únicos se repetirá d vezes ao longo do intervalo de $[0, n - 1]$

Example.

$$k \cdot 4 \bmod 6$$

Para $k \in [0, 5]$:

- Sequência: $\{0, 4, 2, 0, 4, 2)$
- Valores únicos: $\frac{6}{2} = 3$
- Repetição: $d = 2$ vezes

□

6. Outros

Algoritmo de Exponenciação Binária

Para calcular: $2^m = 2^{312}$, ao invés de calcular 312 multiplicações, dividimos o expoente em potências de 2:

$2^{312} = 2^{256} \cdot 2^{32} \cdot 2^{16} \cdot 2^8$

1. $2^8 = \left(\left(2^2\right)^2\right)^2$ - 3 mult.
2. $2^{16} = \left(2^8\right)^2$ - 1 mult.
3. $2^{32} = \left(2^{16}\right)^2$ - 1 mult.
4. $2^{256} = \left(\left(\left(\left(2^{32}\right)^2\right)^2\right)^2\right)$ = 3 mult.
5. Multiplica tudo.

Total de multiplicações: 8 + 4 (n° multiplicações)

No geral: Total de multiplicações = $\lceil \log_2(m) \rceil + (\text{n° de bits 1 em m}) - 2$

Em geral, para um expoente m, $O(n) = \log_2(m)$

7. RSA

RSA

Dado $n = p \cdot q$ (público), com p, q primos, $e \in \mathbb{N}$ (púiblic) e uma mensagem $M < n$, a mensagem criptografada y é:

$$y = M^e \bmod n$$

A chave necessária para descriptografar é:

$$d = e^{-1} \bmod (p-1)(q-1)$$

Onde e^{-1} é o inverso multiplicativo de $e \bmod n$, De modo que:

$$y^d = y^{ee^{-1} \bmod (p-1)(q-1)} = M$$

A segurança no RSA consiste na dificuldade do cálculo dos primos p, q.

Por que a descriptografia do RSA funciona

Por definição:

$$ed = 1 \bmod \varphi(n)$$

Que pode ser reescrito como:

$$ed = 1 + k \cdot \varphi(n)$$

Aplicando a M:

$$M^{ed} = M^{1+k\cdot\varphi(n)} = M^1 \cdot \left(M^{\varphi(n)}\right)^k$$

Sabendo que $mdc(M, n) = 1 \Rightarrow M^{\varphi(n)} = 1 \bmod n$. Assim:

$$M^{ed} = M \cdot (1)^k = M \bmod n$$

Sob estas condições, para uma recuperação perfeita, $\boldsymbol{M}$ **deve ser menor que** $\boldsymbol{n}$.

CAPÍTULO

3

Introdução à Ciência da Computação II

SCC0201

1. Análise de Algoritmos

Comparamos **algoritmos**, não programas: a análise independe de hardware, linguagem e habilidade do programador. $T(n)$ é o custo do algoritmo para uma entrada de tamanho n.

1.1. Notações assintóticas

Definições

Para constantes $c, c_1, c_2 > 0$ e todo $n \geq n_0$:

- $T(n) = \mathcal{O}(f(n)) \Leftrightarrow T(n) \leq c \cdot f(n)$ (limite **superior**)
- $T(n) = \Omega(f(n)) \Leftrightarrow T(n) \geq c \cdot f(n)$ (limite **inferior**)
- $T(n) = \Theta(f(n)) \Leftrightarrow c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ (limite **justo**)

Em $f(n)$ não se escrevem constantes nem termos de menor ordem.

Example.

$T(n) = 3n + 3$:

- $\mathcal{O}(n)$: $3n + 3 \leq cn$ vale com $c = 4$ (para $n \geq 3$). ✓
- $\mathcal{O}(\log n)$: $3n + 3 \leq c \log n$ não vale para nenhum c, pois n cresce mais que $\log n$. ×
- $\Omega(n)$: $3n + 3 \geq cn$ vale com $c = 3$. ✓
- $\Omega(\log n)$: $3n + 3 \geq c \log n$ vale com $c = 1$. ✓

Logo, $T(n) = \Theta(n)$. □

Ordem de crescimento

$$1 < \log n < \log^2 n < n < n \log n < n^2 < n^3 < 2^n < n!$$

Para comparar duas funções, simplifique as duas até chegar nesta escala. Ex.: dividindo por n, comparar $n^{1.5}$ com $n \log n$ vira comparar $n^{0.5}$ com $\log n$. Como $\sqrt{n}$ cresce mais que $\log n$, temos $n \log n < n^{1.5}$.

Séries úteis

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \qquad \sum_{i=0}^{n} 2^i = 2^{n+1} - 1 \qquad \sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1}$$

Melhor, médio e pior caso: $T_{\text{melhor}(n)} \leq T_{\text{médio}(n)} \leq T_{\text{pior}(n)}$. Normalmente se analisa o pior caso.

1.2. Exemplo: subarranjo máximo (divisão e conquista)

Achar o subarranjo contíguo de maior soma. Divide o vetor ao meio; a resposta é o maior entre:

1. o melhor subarranjo da metade esquerda (recursão);
2. o melhor da metade direita (recursão);
3. o melhor que **cruza o meio** (calculado em $\mathcal{O}(n)$).

```
-2 11 -4 13 | -5 -2
esquerda: 11 -4 13 = 20    direita: -2 (ou vazio = 0)    cruzando: 11 -4 13 = 20
resposta: 20
```

Custo: $T(n) = 2T(\frac{n}{2}) + \Theta(n) = \Theta(n \log n)$, contra $\Theta(n^2)$ da força bruta.

1.3. Recorrências

A complexidade de um algoritmo recursivo é dada por uma **recorrência**: uma equação que descreve $T(n)$ em termos de T em entradas menores.

Example.

Fibonacci: $f(0) = 0$, $f(1) = 1$, $f(n) = f(n-1) + f(n-2)$.

```c
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Contando operações: $T(0) = T(1) = 3$ e $T(n) = T(n-1) + T(n-2) + 6$ para $n \geq 2$. □

Método da substituição

Método da substituição

1. **Chute** um limite (por experiência).
2. **Prove por indução** que $T(n)$ não passa dele: assuma que vale para valores menores e mostre que vale para n.

A prova só vale se **uma mesma constante c** servir para todo $n \geq n_0$.

Example.

Fibonacci é $\mathcal{O}(2^n)$. Hipótese: $T(k) \leq c \cdot 2^k$ para $k < n$.

$$T(n) = T(n-1) + T(n-2) + 6 \leq c \cdot 2^{n-1} + c \cdot 2^{n-2} + 6 = \frac{3}{4} c \cdot 2^n + 6$$

Queremos $\frac{3}{4} c \cdot 2^n + 6 \leq c \cdot 2^n$, ou seja, $6 \leq c \cdot \frac{2^n}{4}$, isto é, $c \geq \frac{24}{2^n}$.

O lado direito diminui com n, então $c = 24$ serve para todo $n \geq 0$ (e cobre as bases $T(0) = T(1) = 3$).

✓ □

Example.

Fibonacci não é $\mathcal{O}(n^2)$. A base já falha: $T(0) = 3 \leq c \cdot 0^2 = 0$ é falso. E no passo indutivo:

$$c(n-1)^2 + c(n-2)^2 + 6 \leq cn^2 \Leftrightarrow c(-n^2 + 6n - 5) \geq 6$$

Para $n > 5$ o termo $-n^2 + 6n - 5$ é negativo, então nenhum $c > 0$ funciona. × (De fato, $T(n)$ cresce como 1.618^n.) □

Quando a substituição falha: subtrair um termo menor

$T(n) = 4T(\frac{n}{2}) + n$ é $\Theta(n^2)$ pelo método mestre, mas a hipótese $T(n) \leq cn^2$ não fecha:

$$T(n) \leq 4c\left(\frac{n}{2}\right)^2 + n = cn^2 + n \not\leq cn^2$$

Sobra um $+n$. Solução: **fortalecer** a hipótese subtraindo um termo de menor ordem, $T(n) \leq cn^2 - bn$:

$$T(n) \leq 4\left(c\left(\frac{n}{2}\right)^2 - b\frac{n}{2}\right) + n = cn^2 - 2bn + n \leq cn^2 - bn \Leftrightarrow b \geq 1$$

A indução fecha com $b = 1$. ✓

Método da árvore de recursão

Desenha-se a árvore das chamadas, soma-se o custo de cada nível e depois de todos os níveis.

Example.

$T(n) = T(\frac{n}{2}) + c$: cada nível custa c e o tamanho cai pela metade, então há $\log_2 n + 1$ níveis.

$$T(n) = c(\log_2 n + 1) = \Theta(\log n)$$

□

Método mestre

Teorema mestre

Para $T(n) = aT(\frac{n}{b}) + f(n)$, com $a \geq 1$ e $b > 1$, compare $f(n)$ com $n^{\log_b a}$:

1. Se $f(n) = \mathcal{O}(n^{\log_b a - \varepsilon})$ para algum $\varepsilon > 0$: $T(n) = \Theta(n^{\log_b a})$
2. Se $f(n) = \Theta(n^{\log_b a})$: $T(n) = \Theta(n^{\log_b a} \log n)$
3. Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para algum $\varepsilon > 0$ **e** $af(\frac{n}{b}) \leq cf(n)$ para algum $c < 1$: $T(n) = \Theta(f(n))$

Intuição: quem domina? (1) as folhas da recursão; (2) empate, e cada um dos $\log n$ níveis custa igual; (3) o trabalho de dividir/combinar na raiz.

Recorrência	$n^{\log_b a}$	$f(n)$	Caso	Resultado
$9T(\frac{n}{3}) + n$	n^2	n	1	$\Theta(n^2)$
$T(2\frac{n}{3}) + 1$	$n^0 = 1$	1	2	$\Theta(\log n)$

Recorrência	$n^{\log_b a}$	$f(n)$	Caso	Resultado
$4T(\frac{n}{2}) + n$	n^2	n	1	$\Theta(n^2)$
$4T(\frac{n}{2}) + n^2$	n^2	n^2	2	$\Theta(n^2 \log n)$
$4T(\frac{n}{2}) + n^3$	n^2	n^3	3	$\Theta(n^3)$
$2T(\frac{n}{2}) + n^3$	n	n^3	3	$\Theta(n^3)$
$T(9\frac{n}{10}) + n$	$n^0 = 1$	n	3	$\Theta(n)$

No caso 3, sempre verifique a condição de regularidade. Ex.: para $4T(\frac{n}{2}) + n^3$, $4(\frac{n}{2})^3 = \frac{n^3}{2} \leq cn^3$ com $c = \frac{1}{2} < 1$. ✓

2. Métodos de Ordenação

2.1. Bubble Sort

Percorre o vetor várias vezes trocando vizinhos fora de ordem, até não haver trocas.

- Pior/médio caso: $\mathcal{O}(n^2)$ (dois laços aninhados).
- Melhor caso: $\mathcal{O}(n)$ se parar quando uma passada não fizer trocas.

2.2. Insertion Sort

Mantém um prefixo ordenado à esquerda e insere nele, um a um, os elementos da direita.

```
[10] 2 7 5  ->  [2 10] 7 5  ->  [2 7 10] 5  ->  [2 5 7 10]
```

- Pior caso: $\mathcal{O}(n^2)$. Melhor caso (já ordenado): $\mathcal{O}(n)$.

2.3. Selection Sort (seleção direta)

Busca o menor elemento do restante e o coloca na próxima posição, da esquerda para a direita.

- Sempre $\mathcal{O}(n^2)$ comparações, mas só $\mathcal{O}(n)$ trocas.

2.4. Shell Sort

Insertion sort aplicado a subvetores de elementos à distância h, com h diminuindo até 1. Os elementos andam longe cedo, então a passada final ($h = 1$) tem pouco trabalho.

```
h = 2:  posições pares [10, 7] -> [7, 10]   ímpares [2, 5] já ordenado   =>  7 2 10 5
h = 1:  insertion sort normal                                             =>  2 5 7 10
```

- A complexidade depende da sequência de h. Com a de Knuth ($h_1 = 1$, $h_{i+1} = 3h_i + 1$: 1, 4, 13, 40, ...), é $\mathcal{O}\left(n^{\frac{3}{2}}\right)$.

2.5. Quick Sort

Escolhe um **pivô** x, deixa os menores à esquerda e os maiores à direita (partição) e ordena cada lado recursivamente.

1. Avança i da esquerda até `v[i] >= x`.
2. Recua j da direita até `v[j] <= x`.
3. Se $i \leq j$, troca `v[i]` e `v[j]` e continua.
4. Quando i e j se cruzam, ordena recursivamente as duas partes.

- Pior caso (pivô é sempre o menor/maior): $T(n) = T(n-1) + \mathcal{O}(n) = \mathcal{O}(n^2)$.
- Melhor caso (pivô divide ao meio): $T(n) = 2T(\frac{n}{2}) + \mathcal{O}(n) = \mathcal{O}(n \log n)$.
- Caso médio: $\approx 1.386n \log_2 n - 0.846n = \mathcal{O}(n \log n)$ (Sedgewick e Flajolet, 1996).

2.6. Heap Sort

Um **heap** é uma árvore binária completa em que o pai é maior que os filhos (**max-heap**) ou menor (**min-heap**). Ela é guardada num vetor, com a raiz na posição 0:

Heap em vetor (índice k)

- Pai: $\lfloor \frac{k-1}{2} \rfloor$
- Filho esquerdo: $2k+1$, filho direito: $2k+2$
- Folhas: posições $\lfloor \frac{n}{2} \rfloor$ até $n-1$

1. Constrói um max-heap com o vetor.
2. Troca a raiz (maior) com o último elemento do heap: ele já está na posição final.
3. Diminui o heap em 1 e reorganiza a raiz (desce até o lugar certo).
4. Repete $n-1$ vezes.

- Construir o heap custa $\mathcal{O}(n)$: metade dos nós são folhas (custo 0), $\frac{n}{4}$ descem no máximo 1 nível, $\frac{n}{8}$ no máximo 2, ... e $\sum_i \frac{i}{2^i}$ converge.
- Cada uma das $n-1$ remoções custa $\mathcal{O}(\log n)$. Total: $\mathcal{O}(n \log n)$ **sempre**, sem memória extra.

2.7. Radix Sort

Ordena os números dígito a dígito, **do menos para o mais significativo**, usando uma fila por dígito (0 a 9):

1. Distribui os números nas filas pelo dígito atual.
2. Junta as filas em ordem (0, 1, ..., 9), mantendo a ordem dentro de cada fila (estável).
3. Repete para o próximo dígito.

Ex.: 236 e 235 são separados já no 1º dígito (6 > 5). Custo: $\mathcal{O}(d(n+k))$, onde d é o número de dígitos e k a base.

2.8. Comparação entre os métodos

Tempos relativos (o mais rápido = 1) para vetores de 500 a 30 000 elementos:

	500	5.000	10.000	30.000
Inserção	11,3	87	161	–
Seleção	16,2	124	228	–
Shellsort	1,2	1,6	1,7	2
Quicksort	1	1	1	1
Heapsort	1,5	1,6	1,6	1,6

Figure 15: Ordem aleatória

	500	5.000	10.000	30.000
Inserção	1	1	1	1
Seleção	128	1.524	3.066	–
Shellsort	3,9	6,8	7,3	8,1
Quicksort	4,1	6,3	6,8	7,1
Heapsort	12,2	20,8	22,4	24,6

Figure 16: Já ordenado

	500	5.000	10.000	30.000
Inserção	40,3	305	575	–
Seleção	29,3	221	417	–
Shellsort	1,5	1,5	1,6	1,6
Quicksort	1	1	1	1
Heapsort	2,5	2,7	2,7	2,9

Figure 17: Ordem inversa

Método	**Melhor**	**Pior**	**Quando usar**
Insertion	n	n^2	Até cerca de 20 elementos ou quase ordenado; estável
Selection	n^2	n^2	Quando mover registros é caro (só $\mathcal{O}(n)$ trocas)
Shell	$n \log n$	$n^{\frac{3}{2}}$	Tamanho moderado; sem pior caso ruim
Heap	$n \log n$	$n \log n$	Garantia de $n \log n$ sem memória extra
Merge	$n \log n$	$n \log n$	Garantia de $n \log n$; estável; usa memória extra
Quick	$n \log n$	n^2	Mais rápido na prática; pivô pela mediana de três evita o pior caso

3. Lista 2 - Complexidade

3.1. Ex. 1: menor n com $100n^2 < 2^n$

Basta testar inteiros, pois a exponencial passa a quadrática e não volta mais:

n	$100n^2$	2^n	
14	19 600	16 384	×
15	22 500	32 768	✓

Resposta: $n = 15$.

Solução analítica (função W de Lambert)

W é a inversa de $f(x) = xe^x$: se $xe^x = y$, então $x = W(y)$.

$$2^n = 100n^2 \Rightarrow e^{n \ln 2/2} = 10n \Rightarrow \left(-\frac{\ln 2}{2} n\right) e^{-\frac{\ln 2}{2} n} = -\frac{\ln 2}{20}$$

$$n = -\frac{2}{\ln 2} W\left(-\frac{\ln 2}{20}\right) \approx 14.32 \quad (\text{ramo } W_{-1})$$

O próximo inteiro é $n = 15$.

3.2. Ex. 2: "no mínimo $\mathcal{O}(n^2)$"

$\mathcal{O}$ é um limite **superior**, então "no mínimo $\mathcal{O}(n^2)$" não diz nada: qualquer algoritmo, até um de custo constante, é $\mathcal{O}(n^2)$. Para um limite inferior, usa-se Ω: "o tempo é $\Omega(n^2)$".

3.3. Ex. 3: $2^{n+1} = \mathcal{O}(2^n)$? $2^{2n} = \mathcal{O}(2^n)$?

- $2^{n+1} = 2 \cdot 2^n \leq c \cdot 2^n$ com $c = 2$. **Sim.**
- $2^{2n} = 4^n$, e $4^n \leq c \cdot 2^n \Leftrightarrow 2^n \leq c$, falso para n grande. **Não.**

3.4. Ex. 5: $T(n) = T(n-1) + n$ é $\mathcal{O}(n^2)$

Expandindo a recorrência até a base $T(1) = 1$:

$$T(n) = n + (n-1) + \cdots + 2 + 1 = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$$

CAPÍTULO

4

Equações Diferenciais Ordinárias

SME0340

1. Introdução

- $\frac{dy}{dx} + y = 8x - 7$, onde y é a **variável dependente** e x a **variável independente**.
- **Ordem da EDO** = ordem da maior derivada.
- Notação de derivada: $y^{(n)} = \frac{d^n y}{dt^n}$.

EDO linear

Uma EDO de ordem n é linear se pode ser escrita como

$$F\left(t, y, ..., y^{(n)}\right) = a_0(t)y^{(n)} + a_1(t)y^{(n-1)} + \cdots + a_n(t)y - g(t)$$

Se aparecer y^2, $\ln|y|$, $\sin(y)$, etc., a EDO **não é linear**.

1.1. Problema de Valor Inicial (PVI)

$$\begin{cases} y' = f(t, y) \\ y(t_0) = y_0 \end{cases}$$

Teorema de Peano

Se f é contínua $\forall (t_0, y_0) \in \mathbb{R}^2$, existe **pelo menos uma** solução.

Teorema de Existência e Unicidade (T.E.U.)

Se $f(t, y)$ e $\frac{\partial f}{\partial y}(t, y)$ forem contínuas no intervalo, então o PVI **tem solução única**.

Cuidado: o T.E.U. só vale em PVIs.

Pelo T.E.U., as funções solução **não se interseccionam**:

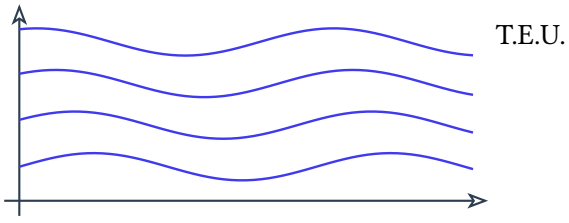


Figure 18: Soluções distintas de um PVI nunca se cruzam.

2. EDOs de 1ª ordem

2.1. EDOs separáveis

EDO separável

Se $f(t,y) = h(t)g(y)$, a EDO $y' = h(t)g(y)$ é separável.

1. **Soluções constantes:** se $y(t) = c$ anula g (isto é, $g(c) = 0$), então $y(t) = c$ é solução constante.
2. **Soluções não constantes:** considerando $y' = g(y)h(t)$ e sendo $y(t)$ solução, segue da definição de diferencial que

$$\mathrm{d}y = y'(t)\,\mathrm{d}t \quad \Rightarrow \quad \mathrm{d}\frac{y}{g(y)} = h(t)\,\mathrm{d}t$$

Esse é o **método de separação de variáveis**. Depois, integra!

2.2. EDOs lineares de 1ª ordem

Forma padrão

$$y' + a(t)y = b(t)$$

1. Calcula a função auxiliar (**fator integrante**):

$$\mu(t) = e^{\int a(t)\,\mathrm{d}t}$$

2. Multiplica a equação por $\mu(t)$:

$$\underbrace{y'e^{\int a(t)\,\mathrm{d}t} + a(t)ye^{\int a(t)\,\mathrm{d}t}}_{\frac{\mathrm{d}}{\mathrm{d}t}\left(ye^{\int a(t)\,\mathrm{d}t}\right)} = b(t)e^{\int a(t)\,\mathrm{d}t}$$

3. Integra dos dois lados (pelo T.F.C.):

$$\int \frac{\mathrm{d}}{\mathrm{d}t}\left(ye^{\int a(t)\,\mathrm{d}t}\right)\mathrm{d}t = \int b(t)e^{\int a(t)\,\mathrm{d}t}\,\mathrm{d}t$$

Solução

$$y\mu(t) = \int b(t)e^{\int a(t)\,\mathrm{d}t}\,\mathrm{d}t + C$$

Não esquecer da constante C!

2.3. EDOs não lineares de 1ª ordem

Equação de Bernoulli

Forma geral

$$y' + a(t)y = b(t)y^n$$

($n > 1$: não linear)

Transformamos em algo linear:

1. Divide por y^n: $y'y^{-n} + a(t)y^{1-n} = b(t)$
2. Substitui $z = y^{1-n}$, logo $z' = (1-n)y^{-n}y'$, ou seja, $\dfrac{z'}{1-n} = y^{-n}y'$.
3. Substitui na original e resolve como se fosse linear de 1ª ordem:

$$\frac{z'}{1-n} + a(t)z = b(t)$$

É linear! (Ao achar z, acha y.)

Equação de Riccati

Forma geral

$$y' + a(t)y + b(t)y^2 = f(t)$$

(Se $f(t) = 0$, é Bernoulli.)

Tomamos $y_1(t)$ uma solução conhecida e fazemos

$$y = y_1 + \frac{1}{z} \quad \Rightarrow \quad y' = y_1' - \frac{z'}{z^2}, \quad y^2 = y_1^2 + \frac{2y_1}{z} + \frac{1}{z^2}$$

Substituindo na original, obtemos:

$$z' = \underbrace{[a(t) + 2b(t)y_1]}_{h(t)}z + b(t)$$

- Riccati se transforma em uma EDO linear de 1ª ordem.
- Ao aplicar Riccati, $f(t)$ zera.

3. Aplicações

3.1. Problemas de crescimento / decaimento

Seja $N = N(t)$ a lei que descreve a evolução de um fenômeno em Δt, e a taxa de variação seja proporcional à quantidade presente:

$$\frac{\mathrm{d}N}{\mathrm{d}t} = kN \quad \to \quad N(t) = Ce^{kt}$$

(k: constante de proporcionalidade.)

3.2. Lei do resfriamento / aquecimento de Newton

A taxa de variação da temperatura no tempo é proporcional à diferença de temperatura entre o corpo e o meio:

$$\frac{\mathrm{d}T}{\mathrm{d}t} = k\big(T_{\text{corpo}} - T_{\text{ambiente}}\big) \quad \to \quad T(t) = Ce^{kt} + T_m$$

($T_{\text{ambiente}} = T_m$ considerado constante.)

3.3. Diluição

A taxa de variação da quantidade de sal no tanque é igual à taxa que entra menos a taxa que sai:

$$\underset{\textit{entrada}}{\alpha \text{ l/min}, b \text{ kg/l}} \qquad \underset{\textit{sal no tanque}}{Q(t)} \qquad \underset{\textit{saída}}{\beta \text{ l/min}}$$

$$\frac{\mathrm{d}Q}{\mathrm{d}t} = \alpha b - C(t)\beta$$

onde $C(t)$ é a concentração no tanque:

$$\frac{\mathrm{d}Q}{\mathrm{d}t} = \alpha b - \underbrace{\frac{Q(t)}{V_0 + \alpha t - \beta t}}_{\text{concentração (kg/l) no tanque}} \beta$$

4. EDOs de 2ª ordem

Homogênea e não homogênea

- $x'' + a(t)x' + b(t)x = 0 \quad \to$ EDO **homogênea** (H)
- $x'' + a(t)x' + b(t)x = f(t) \quad \to$ EDO **não homogênea** (NH)

4.1. EDOs de 2ª ordem homogêneas

T.E.U. para PVI

$$\begin{cases} x'' + a(t)x' + b(t)x = f(t) \\ x(t_0) = \alpha \\ x'(t_0) = \beta \end{cases}$$

Se a, b, f são contínuas, então o PVI possui **solução única**.

Princípio da superposição

Se x_1, x_2 são soluções da EDO (H) de 2ª ordem, então a combinação linear

$$\varphi(t) = C_1 x_1(t) + C_2 x_2(t)$$

também é solução de H.

Obs.: $\{x_1(t), x_2(t)\}$ é a base das soluções da EDO.

Wronskiano

Verifica se x_1, x_2 são L.I. (para formar base):

$$W(x_1, x_2)(t) = \begin{vmatrix} x_1(t) & x_2(t) \\ x_1'(t) & x_2'(t) \end{vmatrix}$$

- $W \neq 0$: L.I.
- $W = 0$: L.D.

Teorema de Abel

$$W(x_1, x_2)(t) = Ce^{-\int a(t)\,\mathrm{d}t}$$

Não depende de t, só de x_1, x_2.

- Evita ter que achar soluções para saber se a base é L.I.
- $a(t)$ é o coeficiente de x' (o coeficiente do 2º maior).
- Para determinar C, calculamos W no ponto t_0.

4.2. EDOs (H, 2ª ordem) com coeficientes constantes

$$x'' + ax' + bx = 0$$

Ao aplicar $x(t) = e^{\lambda t}$ (solução genérica), obtemos a **equação característica**:

Equação característica

$$\lambda^2 + a\lambda + b = 0$$

Achando λ, achamos x_1 e x_2.

- **Caso $\Delta > 0$:** $W \neq 0$ ($\lambda_1 \neq \lambda_2$).

$$x_H(t) = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t}$$

- **Caso $\Delta = 0$:** $\lambda_1 = \lambda_2$, então $W = 0$. Usamos

$$x_1(t) = e^{\lambda t}, \quad x_2(t) = te^{\lambda t}$$

pois $W(x_1, x_2)(t) \neq 0$ (é solução e é L.I.!). Para multiplicidade 3, $x_3 = t^2 e^{\lambda t}$.

Por soma e produto: $\lambda + \lambda = -\frac{a}{1} \quad \Rightarrow \quad \lambda = -\frac{a}{2}$.

- **Caso $\Delta < 0$:** se $\lambda = \alpha \pm i\beta$ ($\beta \neq 0$),

$$x_1(t) = e^{\alpha t}\cos(\beta t), \quad x_2(t) = e^{\alpha t}\sin(\beta t)$$

Obs.: o $\pm$ não importa (usa-se $e^{(\alpha+i\beta)t}$ como solução e a identidade de Euler $e^{i\theta} = \cos\theta + i\sin\theta$, $\theta \in \mathbb{R}$).

$$W(x_1, x_2) = \beta e^{2\alpha t} \neq 0$$

4.3. Teorema de Abel (ordem n)

Teorema de Abel para ordem n

Para EDO de ordem n na forma

$$y^{(n)} + p_{n-1}y^{(n-1)} + \cdots + p_0(x)y = 0$$

$$W(x) = Ce^{-\int a(x)\,\mathrm{d}x}$$

onde $a(x)$ é **sempre** p_{n-1} (se for 0, temos $W(x) = C$).

- Sem achar as soluções, Abel nos dá a forma final do Wronskiano (comportamento).
- Se $W(0) = 0$, Abel garante que $W(t) = 0$ em todos os pontos (o mesmo para $W(0) \neq 0$).
- Se $W(t) = Ct$, por exemplo, então $t = 0$ é um ponto de descontinuidade (função original: $a(t) = \frac{1}{t}$).

Example.

Uso de Abel para calcular o Wronskiano geral. Se temos $y'' + e^x y' + (x^2+1)y = 0$ e $W(0) = 5$, qual é $W(10)$?

$$W(x) = Ce^{-e^x} \quad \text{(por Abel)}$$

$$W(0) = 5 = Ce^{-1} \to C = 5e \to W(x) = 5e \cdot e^{-e^x}$$

que é o Wronskiano em qualquer ponto. □

Example.

Uso de Abel para achar a 2ª solução. Se temos

$$y'' - \left(\frac{x+2}{x}\right)y' + \left(\frac{x+2}{x^2}\right)y = 0$$

e sabemos que $y_1(x) = x$ é solução:

$$W(x) = Cx^2e^x \quad \text{(por Abel)} \quad \text{e} \quad W(x) = y_1y_2' - y_2y_1' = xy_2' - y_2$$

Igualando os dois: $Cx^2e^x = xy_2' - y_2$.

Admite-se $C = 1$ (qualquer múltiplo da solução L.I. também é L.I.):

$$y_2' - \frac{1}{x}y_2 = xe^x \quad \to \text{EDO linear de } 1^{\text{a}} \text{ ordem}$$

□

4.4. Resolução de EDOs não homogêneas de 2ª ordem

$$x'' + a(t)x' + b(t)x = f(t) \quad \text{(NH)}$$

Solução geral

$$x(t) = \underbrace{x_H(t)}_{C_1x_1(t)+C_2x_2(t)} + x_p(t)$$

onde x_H é a solução da EDO (H), sem $f(t)$.

Pois, se aplicarmos $\left[x(t) - x_p(t)\right]$ na EDO, teremos:

$$\underbrace{\left(x''(t) + a(t)x'(t) + b(t)x(t)\right)}_{f(t)} - \underbrace{\left(x''_{p(t)} + a(t)x'_{p(t)} + b(t)x_p(t)\right)}_{f(t)} = 0$$

Teorema da variação dos parâmetros (achar x_p)

Seja $\{x_1(t), x_2(t)\}$ base de soluções da EDO (H) linear de 2ª ordem. Então

$$x_p(t) = x_1(t)\int \frac{W_1(t)f(t)}{W(x_1,x_2)(t)}\,\mathrm{d}t + x_2(t)\int \frac{W_2(t)f(t)}{W(x_1,x_2)(t)}\,\mathrm{d}t$$

onde $W_n(t)$ é o Wronskiano com a coluna n cheia de zeros e o último número 1:

$$W_1(t) = \begin{vmatrix} 0 & x_2(t) \\ 1 & x_2'(t) \end{vmatrix}$$

- Em x_p, não precisa de C (considera-se $C = 0$), pois apenas buscamos uma solução particular.
- Esse teorema se expande para EDOs lineares de ordem n.

5. EDOs de ordem n

T.E.U. (geral)

Se $p_1, p_2, \ldots, p_n, f$ são contínuas no intervalo, então o PVI

$$\begin{cases} x^{(n)} + p_1(t)x^{(n-1)} + \cdots + p_{n-1}(t)x' + p_n(t)x = f(t) \\ x(t_0) = \alpha_0 \\ x'(t_0) = \alpha_1 \\ \vdots \\ x^{(n-1)}(t_0) = \alpha_{n-1} \end{cases}$$

possui **solução única**! (Ordem n: n pontos de definição.)

5.1. Números complexos

Forma polar

$$z = x + iy, \quad z = |z|\ (\cos\theta + i\sin\theta)$$

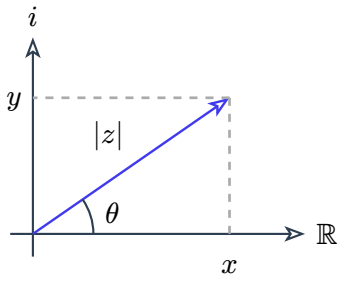


Figure 19: Representação de z no plano complexo.

5.2. Radiciação complexa

2ª fórmula de De Moivre

$$\sqrt[n]{z} = \sqrt[n]{|z|}\left(\cos\left(\frac{\theta + 2k\pi}{n}\right) + i\sin\left(\frac{\theta + 2k\pi}{n}\right)\right)$$

com $k = 0, 1, 2, \ldots, n-1$ (são as raízes!).

De onde vem a 2ª fórmula de De Moivre? As raízes são os vértices de um **polígono regular** inscrito na circunferência de raio $\sqrt[n]{|z|}$. Exemplo: $\sqrt[3]{-1}$.

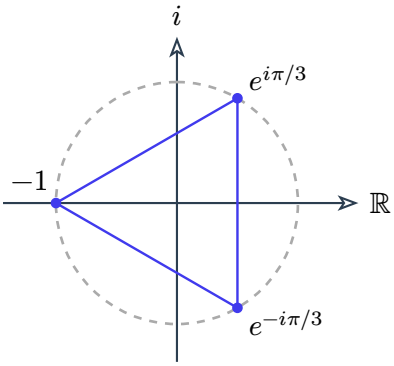


Figure 20: As 3 raízes cúbicas de -1 formam um triângulo equilátero.

5.3. Método das raízes racionais

- Ache os divisores de a_n (coeficiente do maior grau).
- Ache os divisores de a_0 (coeficiente do termo independente).
- Considere positivos e negativos ($\pm$).

Possíveis raízes racionais

$$\frac{D(a_0)}{D(a_n)}$$

Raiz única? Dupla? Se $p'(\lambda) = 0$, é dupla; se $p''(\lambda) = 0$, é tripla; etc.

5.4. Circuito em série RLC

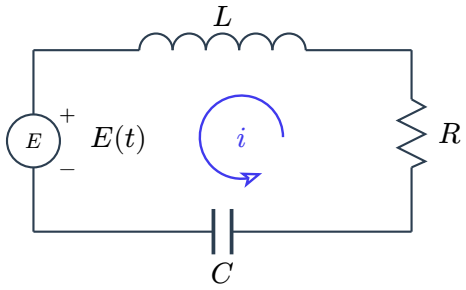


Figure 21: Circuito RLC em série alimentado por $E(t)$.

Lei de Kirchhoff

$$V = V_L + V_R + V_C$$

(a soma das tensões no laço é zero / se conserva).

- **Capacitor:** $Q = CV$ (tensão). Como $i = \frac{\mathrm{d}Q}{\mathrm{d}t}$, temos $\mathrm{d}V = \frac{i\,\mathrm{d}t}{C}$, logo

$$i = C\frac{\mathrm{d}V}{\mathrm{d}t} \quad \text{(corrente)}$$

- **Indutor:** $\varphi = Li$ (corrente). Como $V = \frac{\mathrm{d}\varphi}{\mathrm{d}t}$ (lei de Faraday),

$$V = L\frac{\mathrm{d}i}{\mathrm{d}t} \quad \text{(tensão)}$$

Resumo: $Q = CV$, $\varphi = Li$. Para outras relações, basta derivar.

6. Sistemas de EDOs de 1ª ordem

6.1. O que são?

$$\begin{cases} x_1' = a_{11}(t)x_1 + a_{12}(t)x_2 + \cdots + a_{1n}(t)x_n + g_1(t) \\ \vdots \\ x_n' = a_{n1}(t)x_1 + a_{n2}(t)x_2 + \cdots + a_{nn}(t)x_n + g_n(t) \end{cases}$$

onde $a_{ij}(t)$ e $g_i(t)$ são contínuas em um intervalo $J \subset \mathbb{R}$. Na forma matricial:

$$\begin{pmatrix} x_1'(t) \\ \vdots \\ x_{n(t)}' \end{pmatrix} = \begin{pmatrix} a_{11}(t) & \cdots & a_{1n}(t) \\ \vdots & \ddots & \vdots \\ a_{n1}(t) & \cdots & a_{nn}(t) \end{pmatrix} \begin{pmatrix} x_1(t) \\ \vdots \\ x_n(t) \end{pmatrix} + \begin{pmatrix} g_1(t) \\ \vdots \\ g_n(t) \end{pmatrix}$$

Forma matricial

$$X'(t) = A(t)X(t) + g(t)$$

T.E.U.

Se $a_{ij}(t)$ e $g_i(t)$ são contínuas num intervalo $J \subset \mathbb{R}$, então, dados $t_0 \in J$ e $X_0 \in \mathbb{R}^n$, existe uma **única solução** $X(t)$, definida em J, do PVI:

$$X'(t) = A(t)X(t) + g(t), \quad X(t_0) = X_0 = \begin{pmatrix} x_0^1 \\ \vdots \\ x_0^n \end{pmatrix}$$

- **Sistema homogêneo:** $X' = AX$
- **Sistema não homogêneo:** $X' = AX + g(t)$

Matriz solução

$X(t)_{n\times n}$ é matriz solução do sistema $X' = A(t)X$ se cada coluna de $X(t)$ for uma solução desse sistema.

Example.

$$X(t) = \begin{pmatrix} e^t & 0 \\ 0 & e^{2t} \end{pmatrix}$$

é solução de $X' = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} X$. Derivando a 1ª coluna:

$$X_1'(t) = \begin{pmatrix} e^t \\ 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \underbrace{\begin{pmatrix} e^t \\ 0 \end{pmatrix}}_{X_1(t)}$$

(e o mesmo para X_2).

Obs.: chamando as colunas de $X(t)$ de $X_i(t)$, dizemos, por exemplo, que $X_i'(t) = AX_i(t)$. □

Matriz fundamental

Matriz solução com $\det X(t) \neq 0 \ \forall t \in J$ (colunas L.I.).

Solução geral

Se $X(t)$ é matriz fundamental de $X' = A(t)X$, então a solução geral é dada por

$$X(t) = X(t)C, \quad C = \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix} \in \mathbb{R}^n \text{ (vetor constante)}$$

Fórmula de Jacobi-Liouville

$$\det X(t) = \det X(t_0) e^{\int_{t_0}^{t} \operatorname{tr}(A(s))\,\mathrm{d}s}$$

($t_0 = 0$, por exemplo), onde $\operatorname{tr}(A(s))$ é o traço de $A(s)$ (soma dos elementos da diagonal principal).

Serve para verificar se é a matriz fundamental: é o teorema de Abel melhorado.

6.2. Solucionando sistemas $X' = AX$ (homogêneos)

Basta achar autovalores/autovetores de

$$(A - \lambda I)\vec{v} = \vec{0} \quad \to \quad \det(A - \lambda I) = 0$$

(descobrir os autovalores pelo **polinômio característico**). Para achar os autovetores, basta substituir λ em $(A - \lambda I)\vec{v} = \vec{0}$.

Matriz fundamental

$$X(t) = \left(e^{\lambda_1 t} v_1, \ldots, e^{\lambda_n t} v_n\right) \quad \to \quad X_H(t) = X(t)\vec{C}$$

($\vec{C}$: vetor de constantes).

Autovalores complexos de A

Com $\lambda = \alpha + i\beta$ e $v = v_1 + iv_2$:

$$e^{\lambda t} v = e^{(\alpha+i\beta)t}(v_1 + iv_2) = e^{\alpha t}(\cos(\beta t) + i\sin(\beta t))(v_1 + iv_2)$$

$$= \underbrace{e^{\alpha t}(v_1 \cos(\beta t) - v_2 \sin(\beta t))}_{X_1 = \operatorname{Re}(X(t)) \quad \text{(solução 1)}} + \underbrace{ie^{\alpha t}(v_1 \sin(\beta t) + v_2 \cos(\beta t))}_{X_2 = \operatorname{Im}(X(t)) \quad \text{(solução 2)}}$$

Obs.: sim, tem que achar os autovetores depois.

Autovalores repetidos

- $m_a(\lambda)$: grau de λ no $p(\lambda)$ (multiplicidade algébrica).
- $m_g(\lambda)$: nº de autovetores L.I. que λ gera (multiplicidade geométrica).

Casos:

- $m_g(\lambda) = m_a(\lambda) \to \lambda$ gera exatamente $m_a(\lambda)$ autovetores L.I.
- $m_g(\lambda) < m_a(\lambda) \to \lambda$ gera $m_g(\lambda)$ autovetores L.I., mas precisamos de $m_a(\lambda)$. Temos que buscar os autovetores restantes:

$$
\begin{aligned}
(A-\lambda I)\vec{v}_1 &= \vec{0} \\
(A-\lambda I)\vec{v}_2 &= \vec{v}_1 \\
(A-\lambda I)\vec{v}_3 &= \vec{v}_2 \\
&\vdots \\
(A-\lambda I)\vec{v}_n &= \vec{v}_{n-1}
\end{aligned}
\qquad
\begin{aligned}
X_1(t) &= e^{\lambda t} v_1 \\
X_2(t) &= e^{\lambda t}(v_1 t + v_2) \\
X_3(t) &= e^{\lambda t}\left(v_1 \frac{t^2}{2!} + v_2 t + v_3\right) \\
&\vdots \\
X_n(t) &= e^{\lambda t} \sum_{i=1}^{n} \frac{t^{n-i}}{(n-i)!} v_i
\end{aligned}
$$

6.3. Sistemas lineares não homogêneos

$$X' = A(t)X + f(t)$$

$$X(t) = \underbrace{X(t)C}_{\text{matriz fundamental de } X'=A(t)X} + \underbrace{X_p(t)}_{\text{solução particular}}$$

Variação dos parâmetros

$$X_p(t) = X(t) \int X^{-1}(t) f(t)\, \mathrm{d}t$$

($\int$ = integral elemento a elemento; $X(t)$ é a matriz fundamental obtida do homogêneo).

Adicional: inversa 2×2

Se $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ e $\det M = ad - bc \neq 0$, então

$$M^{-1} = \frac{1}{\det M} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

(diagonal principal troca, diagonal secundária nega).

6.4. EDO de ordem $k \geq 2$ como sistema

Podemos transformar uma EDO de ordem $n \geq 2$ em um sistema de n equações diferenciais de 1ª ordem.

Example.

$y^{(5)} - 2y^{(4)} + 4y''' - 6y'' + 7y' = 0$ (escrito com $y_0 = y$):

$$
\begin{cases}
y_0' = y_1 \\
y_1' = y_2 \\
y_2' = y_3 \\
y_3' = y_4 \\
y_4' - 2y_3 + 4y_2 - 6y_1 + 7y_0 = 0
\end{cases}
\qquad
\underbrace{\begin{pmatrix} y_0' \\ y_1' \\ y_2' \\ y_3' \\ y_4' \end{pmatrix}}_{Y'} = \underbrace{\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ -7 & 6 & -4 & 2 & 0 \end{pmatrix}}_{A(t)} \underbrace{\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix}}_{Y}
$$

□

7. Transformada de Laplace

Definição

$$\mathcal{L}\{f(t)\} = F(s) = \int_0^{+\infty} e^{-st} f(t)\,\mathrm{d}t$$

(com $f(t)$ definida para $t \geq 0$ e a integral convergindo). F é a transformada da função f.

Usamos a transformada para trocar uma **equação diferencial** $f\left(y^{(n)}, ..., y, t\right) = 0$ por uma **equação algébrica** $f(s) = 0$ em um domínio diferente. Ao manipular $f(s) = 0$ e aplicar a transformada inversa, temos a resposta $y(t)$.

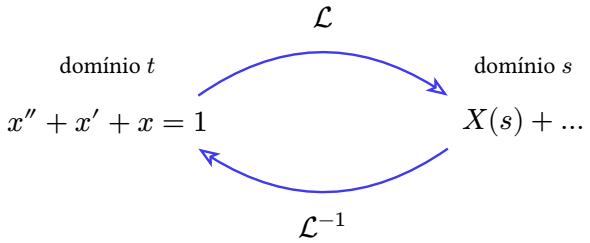


Figure 22: A transformada leva o problema ao domínio s; a inversa traz de volta.

7.1. Transformadas básicas

$\mathcal{L}\{1\} = \frac{1}{s}, \quad s > 0$	$\mathcal{L}\{e^{kt}\} = \frac{1}{s-k}, \quad s > k$
$\mathcal{L}\{t^n\} = \frac{n!}{s^{n+1}}, \quad s > 0$	$\mathcal{L}\{\sin(kt)\} = \frac{k}{s^2+k^2}, \quad s > 0$
	$\mathcal{L}\{\cos(kt)\} = \frac{s}{s^2+k^2}, \quad s > 0$

7.2. Condição para convergência

Ordem exponencial

f é de ordem exponencial se $\exists c \in \mathbb{R}, M > 0, T \geq 0$ tais que

$$|f(t)| \leq Me^{ct} \quad \forall t \geq T$$

(conseguimos limitar a função por uma exponencial). Ex.: $f(t) = t$, $f(t) = \sin(t)$.

Contínua por partes

Uma função é contínua por partes em $I \subset \mathbb{R}$ se, em qualquer subintervalo de I, há apenas um número finito de descontinuidades e toda descontinuidade é de **primeira espécie** (existem os limites laterais e eles tendem a um número).

(2ª espécie: um dos lados vai para $\pm\infty$.)

Se $f(t)$ é de ordem exponencial e contínua por partes, ela converge em Laplace.

7.3. Propriedades

Teorema (Laplace é linear)

$$\mathcal{L}\{\alpha f(t) + \beta g(t)\} = \alpha\mathcal{L}\{f(t)\} + \beta\mathcal{L}\{g(t)\}$$

Teorema (deslocamento de domínio)

$$\mathcal{L}\{e^{ct} f(t)\} = F(s-c) \quad (s > a + c)$$

Teorema ($t^n f(t)$)

$$\mathcal{L}\{t^n f(t)\} = (-1)^n F^{(n)}(s)$$

onde $F^{(n)}(s)$ é a derivada $\dfrac{\mathrm{d}^n}{\mathrm{d}s^n}$ de $\mathcal{L}\{f(t)\}$.

Fórmula da derivada

Sejam $f, f', \ldots, f^{(n-1)}$ contínuas e $f^{(n)}$ contínua por partes em $[0, A]$. Então

$$\mathcal{L}\{f^{(n)}(t)\} = s^n\mathcal{L}\{f(t)\} - \sum_{i=1}^{n} s^{n-i} f^{(i-1)}(0)$$

Os expoentes de s vão de s^{n-1} a s^0 e as derivadas de $f^{(0)}(0) = f(0)$ a $f^{(n-1)}(0)$ (contrários).

Obs.: lembre-se de aplicar no ponto zero.

Fórmula da derivada inversa

$$\mathcal{L}^{-1}\left\{\frac{\mathrm{d}}{\mathrm{d}s}F(s)\right\} = -tf(t)$$

7.4. Transformada inversa

Consiste em voltar: $\mathcal{L}^{-1}\{F(s)\}$. Buscar funções elementares no domínio de s (seja manipulando ou fazendo frações parciais).

Teorema (existência da inversa)

Se $f(t)$ for contínua por partes em $[0, \infty)$ e de ordem exponencial para $t \geq T$, então:

$$\lim_{s\to\infty} \mathcal{L}\{f(t)\} = \lim_{s\to\infty} F(s) = 0$$

No infinito, $F(s)$ tem que dar zero.

7.5. Função degrau unitário

Permite modelar mudanças bruscas, descontinuidades e forças que ligam/desligam em sistemas dinâmicos descritos por EDOs. Funciona como um "interruptor": $U(t-a)$ liga quando $t=a$ e permanece em $t>a$.

Degrau unitário

Para $a>0$:

$$U(t-a)=\begin{cases}0, & 0\le t<a \quad \text{(desligado)}\\ 1, & t\ge a \quad \text{(ligado)}\end{cases}$$

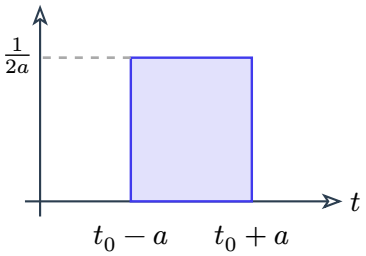
Transformada do degrau

$$\mathcal{L}\{f(t-a)U(t-a)\}=e^{-as}F(s), \quad a>0$$

onde $F(s)=\mathcal{L}\{f(t)\}$.

7.6. Função delta de Dirac

Permite modelar **forças impulsivas**, que aplicam uma intensidade imensa em um intervalo de tempo praticamente instantâneo. "Impacto".



Pulso δ_a

$$\delta_a(t-t_0)=\begin{cases}0, & 0\le t<t_0-a\\ \frac{1}{2a}, & t_0-a\le t<t_0+a \quad a>0,\ t_0>0\\ 0, & t\ge t_0+a\end{cases}$$

$$\int_0^{+\infty}\delta_a(t-t_0)\,\mathrm{d}t=1, \quad \lim_{a\to 0}\delta_a(t-t_0)=0, \quad \lim_{a\to 0}\int_0^{+\infty}\delta_a(t-t_0)\,\mathrm{d}t=1$$

Vamos definir uma função impulso unitária que funciona como impulso de tamanho 1 em $t=t_0$:

Delta de Dirac

1. $\delta(t-t_0)=0$ se $t\neq t_0$
2. $\displaystyle\int_0^{+\infty}\delta(t-t_0)\,\mathrm{d}t=1$

$$\mathcal{L}\{\delta(t-t_0)\}=\lim_{a\to 0}\mathcal{L}\{\delta_a(t-t_0)\}=e^{-st_0}$$

7.7. Convolução (importante)

Convolução

$$(f * g)(t) = \int_0^t f(\tau)g(t-\tau)\,\mathrm{d}\tau \quad \left(= \int_0^t f(t-\tau)g(\tau)\,\mathrm{d}\tau\right)$$

Teorema da convolução

Sejam f, g contínuas por partes em $[0, +\infty)$:

$$\mathcal{L}\{(f * g)(t)\} = F(s) \cdot G(s)$$

$$(f * g)(t) = \mathcal{L}^{-1}\{F(s) \cdot G(s)\}$$

Permite separar funções na transformada inversa!

Example.

$$\begin{cases} y'' - 6y' + 9y = t^2 e^{3t} \\ y(0) = 2 \\ y'(0) = 6 \end{cases}$$

Aplicando $\mathcal{L}$ dos dois lados e a fórmula da derivada (com $Y = \mathcal{L}\{y\}$):

$$[s^2 Y - sy(0) - y'(0)] - 6[sY - y(0)] + 9Y = \underbrace{\frac{2!}{(s-3)^3}}_{\text{deslocamento de domínio}}$$

Organizando:

$$Y(s) = \frac{2}{(s-3)^5} + \frac{2}{s-3}$$

Aplicando $\mathcal{L}^{-1}$: $\dfrac{2}{(s-3)^5} \to \dfrac{1}{12}e^{3t}t^4$ (t^n com domínio deslocado em $a = 3$) e $\dfrac{2}{s-3} \to 2e^{3t}$:

$$y(t) = \frac{t^4 e^{3t}}{12} + 2e^{3t}$$

Sempre que tiver termo elevado a alta potência, vai ser um t^n. □

8. Anotações da Lista

8.1. Como resolver $\sin()\cos()$?

Não resolve: separa.

Produto de senos e cossenos

$$\sin(2t) = 2\sin(t)\cos(t)$$

$$\sin(A)\cos(B) = \frac{1}{2}(\sin(A-B) + \sin(A+B))$$

$$\cos(A)\cos(B) = \frac{1}{2}(\cos(A-B) + \cos(A+B))$$

$$\sin(A)\sin(B) = \frac{1}{2}(\cos(A-B) - \cos(A+B))$$

8.2. Transformadas em cadeia

$$te^{at}\sin(bt) = F(s-a), \quad F = \mathcal{L}\left\{ t\underbrace{\sin(bt)}_{G} \right\} = (-1)^1 G'(s)$$

$$= \frac{2(s-a)b}{((s-a)^2 + b^2)^2}$$

8.3. Interruptor (U)

$$f(t) = \begin{cases} t, & 0 \le t < 1 \\ 1, & t \ge 1 \end{cases} \quad \rightarrow \quad f(t) = t\underbrace{(1 - U(t-1))}_{[0,1)} + 1 \cdot \underbrace{U(t-1)}_{[1,\infty)}$$

$$f(t) = t - tU(t-1) + U(t-1) \quad \text{(precisamos de } (t-1))$$

$$= t - (t-1+1)U(t-1) + U(t-1)$$

Obs.: se $f(t) = 2$, então $f(t-a) = 2$ (constante).

8.4. Periodicidade

$$\mathcal{L}\{\sin(t)U(t-2\pi)\} = \mathcal{L}\{\sin(t-2\pi)U(t-2\pi)\}$$

$$\cos(2t) + \cos(2t)U(t-\pi) = \cos(2t) + \cos(2t+2\pi-2\pi)U(t-\pi)$$

$$= \cos(2t) + \cos(2(t-\pi))U(t-\pi) \quad \overset{\mathcal{L}}{\rightarrow} \quad \ldots + e^{-\pi s}\frac{s}{s^2+4}$$

8.5. Outras anotações

- **Se tem potência, é t^n:** $\dfrac{4}{(s-1)^3} \quad \overset{\mathcal{L}^{-1}}{\rightarrow} \quad 2e^t t^2$
- **Convolução vale propriedade associativa:** $\mathcal{L}\{f * g * h\} = F \cdot G \cdot H$
- **Convolução é comutativa:** $f * g = g * f$
- **Como saber se a transformada é válida?** $\lim_{s\to\infty} F(s) = 0$
- **Função periódica:** $f(t) = f(t+T) = f(t+2T)$
- **Regra da derivada:** $s^n Y - s^{n-1}y(0) - \ldots$ (começa do zero e continua caindo).

- **Função delta:** $\delta(t-a) \overset{\mathcal{L}}{\to} e^{-as}$

Frações parciais

- **Termos simples:** $(x-1)(x+2) \to \dfrac{A}{x-1} + \dfrac{B}{x+2}$
- **Termo 1 elevado:** $(x-1)^2 \to \dfrac{A}{x-1} + \dfrac{B}{(x-1)^2}$
- **Termo 2 elevado sem raízes reais:** $(x^2+4) \to \dfrac{Ax+B}{x^2+4}$

Obs.:

$$\frac{P(s)}{(s^2+2)^2} = \frac{As+B}{s^2+2} + \frac{Cs+D}{(s^2+2)^2}$$

Example.

Achar frações parciais por substituição (bem mais fácil!):

$$\frac{s+1}{(s-1)(s+2)} = \frac{A}{s-1} + \frac{B}{s+2}$$

$$s+1 = A(s+2) + B(s-1)$$

- $s=1$: $2 = 3A \to A = \frac{2}{3}$
- $s=-2$: $-1 = -3B \to B = \frac{1}{3}$

□

CAPÍTULO

5

Estatística

SME0123

1. Probabilidade básica

União de eventos

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Mutuamente Exclusivo

A, B mutuamente exclusivos:

$$P(A \cap B) = 0$$

Probabilidade condicional

Probabilidade de A dado que ocorreu B:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \Leftrightarrow P(A \cap B) = P(A|B)P(B)$$

Independência

$$P(A|B) = P(A) \rightarrow P(A \cap B) = P(A)P(B)$$

AND/OR

$$P(A \text{ e } b) \rightarrow P(A \cap B)$$

$$P(A \text{ ou } b) \rightarrow P(A \cup B)$$

Espaço amostral

Se há apenas A, B e C em Ω, então:

$$P(A) + P(B) + P(C) = 1$$

Probabilidade total

$$P(A) = P(A \cap B) + P\left(A \cap \overline{B}\right)$$

$$P(A) = P(A \mid B)P(B) + P\left(A \mid \overline{B}\right)P\left(\overline{B}\right)$$

$$P(A) = P(A \cap B \cap C) + P\left(A \cap \overline{B} \cap C\right) + P\left(A \cap B \cap \overline{C}\right) + P\left(A \cap \overline{B} \cap \overline{C}\right)$$

Complemento

$$P(A) = 1 - P\left(\overline{A}\right)$$

$$P\left(\overline{A} \mid B\right) = 1 - P(A \mid B)$$

Bayes (Troca)

Se $\cup_{k=0}^{N} A_i = \Omega$, então:

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum P(B|A_i)P(A_i)}$$

(São intersecções: $P(A_i \cap B)$

O segredo para trabalhar com exercícios de probabilidade é, primeiro, extrair $P(A)$, $P(A|B)$, $P(A \cap B)$, montar as proposições, e trabalhar com as propriedades.

2. Funções de probabilidade

Variável aleatória

Variável aleatória é uma função $X(\omega)$ que associa a cada elemento $\omega \in \Omega$ um número real $x = x(\omega)$.

$$\Omega = \{S, S, S, N, N\} \rightarrow X(\omega) = \{1, 1, 1, 0, 0\}$$

Ela pode ser discreta ou contínua (tempo de vida de objeto, em horas).

Example.

Ex: Duas pessoas jogam 3 moedas cada. Qual a probabilidade de obter o mesmo número de caras?

(k)

$$X(\Omega) = \begin{cases} k = 0 \rightarrow 1 \text{ poss.} \rightarrow k = \frac{1}{8} \\ k = 1 \rightarrow 3 \text{ poss.} \rightarrow k = \frac{3}{8} \\ k = 2 \rightarrow 3 \text{ poss.} \rightarrow k = \frac{3}{8} \\ k = 3 \rightarrow 1 \text{ poss.} \rightarrow k = \frac{1}{8} \end{cases}$$

Total: 8 possibilidades

$$P(A = B) = P(k_a = 0 \cap k_b = 0) + P(k_a = 1 \cap k_b = 1) +$$

$$+P(k_a = 2 \cap k_b = 2) + P(k_a = 3 \cap k_b = 3)$$

A e B são independentes, logo: $P(A \cap B) = P(A)P(B)$

$$P(A = B) = \left(\frac{1}{8}\right)^2 + \left(\frac{3}{8}\right)^2 + \left(\frac{3}{8}\right)^2 + \left(\frac{1}{8}\right)^2$$

□

Função massa de probabilidade ou f.m.p. (discreta)

Associa a cada variável aleatória discreta um valor.

$$p_x(x_i) = P(X(w_i))$$

Deve satisfazer:

1. $p_{x(x_i)} > 0$ para todo x_i
2. $\sum p_{x(x_i)} = 1$

Função densidade de probabilidade ou f.d.p. (contínua)

Associa a cada variável aleatória um valor.

$$f_x(x_i) = P(X(w_i))$$

Deve satisfazer:

1. $f_{x(x)} \geq 0$ para todo $x \in R_x$
2. $\int_{-\infty}^{+\infty} f_x(x)dx = 1$
3. $P(a < X < b) = \int_a^b f_x(x)dx$

Propriedades:

- $P(x > a) = P(x \geq a) = \int_a^{+\infty} f_x(x)dx$
- $P(x \geq a) = 1 - P(X < a)$

Função distribuição acumulada

Caso discreto:

$$F_x(x) = P(X \leq x) = \sum_{x_i \leq x} p_x(x_i)$$

Caso contínuo (é a primitiva):

$$F_x(x) = P(X \leq x) = \int_{-\infty}^{x} f_x(t)dt$$

Consequência imediata:

$$P(a < X < b) = \int_a^b f_x(x)dx = F_x(b) - F_x(a)$$

Esperança Matemática/Valor médio

Seja X uma variável aleatória. No caso discreto:

$$\mu = E(X) = \sum xp_x(x)$$

($p_x(x)$ virá das distribuições). Caso contínuo:

$$\mu = E(X) = \int_{-\infty}^{+\infty} xf_x(x)dx$$

Propriedades:

1. Se $X = a$ (constante), então $E(X) = E(a) = a$
2. $E(E(X)) = E(X)$
3. $E(aX \pm bY) = aE(X) \pm bE(Y)$

Variância

$$\sigma^2 = \text{Var}(X) = E((X - E(X))^2)$$

$$\sigma^2 = E(X^2) - [E(X)]^2$$

Obs: $E(X^2) = \sum x^2 p_x(x)$ ou $\int_{-\infty}^{+\infty} x^2 f_x(x)dx$

Propriedades:

1. Se $X = a$ (constante), então $\text{Var}(X) = \text{Var}(a) = 0$
2. $\text{Var}(X \pm a) = \text{Var}(X)$
3. Se X e Y são independentes: $\text{Var}(aX \pm bY) = a^2\ \text{Var}(X) + b^2\ \text{Var}(Y)$

A raiz quadrada da variância é o desvio padrão.

3. Distribuições

3.1. Distribuições Discretas

Distr. Uniforme Discreta

A variável X assume cada um de seus valores com igual probabilidade.

$$p_x(x) = \begin{cases} \frac{1}{k},\ x \in \{x_1, ..., x_k\} \\ 0,\ \text{caso contrário} \end{cases}$$

Notação: $X \sim U_d(k)$

Distr. de Bernoulli

A variável X assume apenas dois valores:

$$\begin{cases} 1, \text{ se sucesso} \\ 0, \text{ se fracasso} \end{cases}$$

$$p_x(x) = \begin{cases} p^x(1-p)^{1-x},\ x \in \{0, 1\} \\ 0,\ \text{caso contrário} \end{cases}$$

Notação: $X \sim Ber(p)$

$$\mu = p$$

$$\sigma^2 = p(1-p)$$

Distr. Binominal

Considere **M eventos de Bernoulli independentes**, todos com a **mesma probabilidade de sucesso p**.

$$p_x(x) = \begin{cases} \binom{M}{x} p^x (1-p)^{M-x},\ x \in \{0, 1, ..., M) \\ 0,\ \text{caso contrário} \end{cases}$$

(Obs: diferente de Bernoulli, usa-se $x \in \{0, 1, ..., M\}$ na potência!)

Notação: $X \sim Bin(M, p)$

$$\mu = Mp$$

$$\sigma^2 = Mp(1-p)$$

Example.

O fabricante indica que a taxa de equipamentos em perfeito estado é 97%.

a) Seleciona-se ao acaso 20 destes itens. Qual a probabilidade de que haja **pelo menos um** item defeituoso nesses 20?

1. Probabilidade de ter defeito: $1 - 0.97 = 0.03$
2. Pelo menos um: $p_x(X \geq 1) = 1 - p(X = 0)$
3. $p_x(0) = 1 - \binom{20}{0}(0.03)^0(1-0.03)^{20-0} = 1 - 0.54 = 0.46 = 46\%$

b) Selecionando-se aleatoriamente 20 itens em cada um de 10 carregamentos, qual a probabilidade de que haja 3 carregamentos com pelo menos um item defeituoso?

1. Agora que temos a probabilidade de um único caregamento, **aplica-se distribuição binomial novamente com o novo p**.
2. $p_x(3) = \binom{10}{3}(0.46)^3(1-0.46)^{10-3} = 0.16 = 16\%$

□

Distr. Geométrica

Considere uma sequência de eventos de Bernoulli independentes, todos com a **mesma probabilidade de sucesso p**. Seja a variável X o número de fracassos anteriores ao primeiro sucesso.

$$p_x(x) = \begin{cases} p(1-p)^x,\ x \in \{0, 1, ...\} \\ 0,\ \text{caso contrário} \end{cases}$$

Notação: $X \sim Geo(p)$

$$\mu = \frac{1-p}{p}$$

$$\sigma^2 = \frac{1-p}{p^2}$$

Example.

Um pesquisador está realizando experimentos químicos independentes e sabe que a probabilidade de que cada experimento apresente uma reação positiva é 0.3. Qual a probabilidade de que menos de 3 reações negativas ocorram antes da primeira positiva?

1. $p_x(X \leq 2) = \sum_{x=0}^{2} 0.3(1-0.3)^x = 0.66 = 66\%$

□

Distr. Binomial Negativa

Considere eventos de Bernoulli independentes, todos com a mesma probabilidade de sucesso p. Seja a variável X como o número de fracassos anteirores ao r-ésimo sucesso.

$$p_x(x) = \begin{cases} \binom{x+r-1}{r-1} p^r (1-p)^x, \ x \in \{0, 1, ...\} \\ 0, \text{ caso contrário} \end{cases}$$

Notação: $X \sim BN(r, p)$

$$\mu = r\left(\frac{1-p}{p}\right)$$

$$\sigma^2 = r\left(\frac{1-p}{p^2}\right)$$

Example.

Em uma série do campeonato, o time que ganhar 4 em 7 jogos será o vencedor. Se a probabilidade do time A ganhar de B é 55% e A e B se enfrentarão em uma série de 7 jogos, qual a probabilidade de que A vença a série em 6 jogos?

1. r-ésimo sucesso: 4
2. Fracassos: 2 em 6 jogos.
3. $p_{x(2)} = \binom{2+4-1}{4-1} 0.55^4 (1-0.55)^2 = 0.18 = 18\%$

□

Distr. Hipergeométrica

Considere um conjunto de n objetos dos quais m são do tipo I e $n-m$ são do tipo II (complementos). Para um sorteio de $r < n$ objetos, feito ao acaso e sem reposição, defina X como o número de objetos do tipo I selecionados.

$$p_x(x) = \frac{\binom{m}{x}\binom{n-m}{r-x}}{\binom{n}{r}}$$

$$\max\{0, r-(n-m)\} \leq x \leq \min\{r, m\}$$

Notação: $x \sim Hgeo(m, n, r)$

$$\mu = r\frac{m}{n}$$

$$\sigma^2 = \frac{rm(n-m)(n-r)}{n^2(n-1)}$$

Example.

Considere que em um lote de 20 peças existam 4 defeituosas. Selecionando 5 dessas peças, sem reposição, qual a probabilidade de escolher 2 defeituosas?

1. $m = 4 \rightarrow n = 16$
2. $r = 5$
3. $p_x(2) = \frac{\binom{4}{2}\binom{16-4}{5-2}}{\binom{16}{5}} = 0.3 = 30\%$

□

Distr. Poisson

Muito usada quando se deseja contar o número de eventos de certo tipo que ocorrem em um certo período de tempo ou superfície/volume. Utiliza-se o parâmetro $\lambda > 0$ (média).

$$p_x(x) = \begin{cases} \frac{e^{-\lambda}\lambda^x}{x!}, \ x \in \{0, 1, ...\} \\ 0, \text{ caso contrário} \end{cases}$$

Notação: $X \sim Poi(\lambda)$

$$\mu = \lambda$$

$$\sigma^2 = \lambda$$

Example.

Uma central telefônica recebe, em média, cinco chamadas p/min. Supondo que a distribuição Poisson seja adequada nessa situação, obtenha a probabilidade de que a central telefônica receba no máximo duas chamadas durante um intervalo de um minuto.

1. $\lambda = 5$
2. $p_{x(X \leq 2)} = \sum_{x=0}^{2} \frac{e^{-5}.5^x}{x!} = 0.12 = 12\%$

□

Obsevações

1. Se $X_1, ..., X_n$ são variáveis aleatórias indepentes e $X_i \sim Poi(\lambda_i)$, então
$$Y = X_1 + ... + X_n \sim Poi(\lambda_1 + ... + \lambda_n)$$
2. Se $X \sim Bin(M, p)$, com $M \gg p$, pode-se aproximar para Poisson com $\lambda = Mp$
3. $E(x) = Var(x) = \lambda$

3.2. Distribuições Contínuas

Distr. Uniforme

Uma variável aleatória contínua X tem distribuição uniforme no intervalo $[\alpha, \beta], a, b \in \mathbb{R}$ se:

$$f_x(x) = \begin{cases} \frac{1}{\beta - \alpha}, \ \alpha \leq x \leq \beta \\ 0, \text{ caso contrário} \end{cases}$$

Obs: Área = $(\beta - \alpha) \cdot \frac{1}{\beta - \alpha} = 1$

Notação: $X \sim U(\alpha, \beta)$

$$\mu = \frac{\alpha + \beta}{2}$$

$$\sigma^2 = \frac{(\beta - \alpha)^2}{12}$$

Distr. Exponencial

Uma variável aleatória contínua X tem distribuição exponencial com parâmetro $\lambda > 0$ (média) se:

$$f_x(x) = \begin{cases} \frac{1}{\lambda} e^{-\frac{x}{\lambda}}, \ x \geq 0 \\ 0, \text{ caso contrário} \end{cases}$$

Notação: $X \sim Exp(\lambda)$

$$\mu = \lambda$$

$$\sigma^2 = \lambda^2$$

Distr. Normal/Gaussiana

Uma variaüel aleatória contínua X tem distribuição normal com μ (média) e σ^2 (variância) se:

$$f_x(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

$$-\infty < x < \infty$$

Notação: $X \sim N(\mu, \sigma^2)$

Propriedades:

1. A distribuição é simétrica em relação à média. $f_x(\mu - x) = f_x(\mu + x)$.
2. A $f.d.a$ é uma integral sem solução analítica. Calculam-se as probabilidades com auxílio de tabela.

Distr. Normal-padrão

Se $X \sim N(\mu, \sigma^2)$, então a variável aleatória Z;

$$Z = \frac{X - E(X)}{\sqrt{Var(X)}} = \frac{X - \mu}{\sigma}$$

Terá distribuição normal com **média 0 e variância 1**, $Z = N(0, 1)$.

Aprox. da binomial pela normal

Se $X \sim Bin(M, p)$, sabe-se que:

$$E(X) = \mu = Mp$$

$$Var(X) = \sigma^2 = Mp(1-p)$$

Seja $Y \sim N(\mu, \sigma^2)$, com os valores anteriores. Assim, aproxima-se a binomial pela normal.

$$X \sim Bin(M, p) \approx N(\mu, \sigma^2)$$

A aproximação é boa quando $Mp(1-p) \geq 3$.

Teorema do limite central

À medida que o tamanho da amostra (n) aumenta, a distribuição da média amostral aproxima-se de uma distribuição normal, independentemente da forma da distribuição original da população.

- A média da distribuição amostral é igual à media da população original.
- O desvio padrão é dado por $\frac{\sigma}{\sqrt{n}}$

> ***Example.***
>
> Amostra com n=25 de população com $N(80, 25)$. ($\sigma = 5$)
>
> Amostra com n=36 de população com $N(75, 9)$. ($\sigma = 3$)
>
> Qual a probabilidade da média amostral em n=25 exceder a média amostral em n=26 por no

Distribuição de Weibull

Uma variável aleatória X tem distribuição de Weibull se, para $\lambda > 0$ (média):

$$f_x(x, \alpha, \beta) = \begin{cases} \alpha\beta x^{\beta-1} e^{-\alpha x^{\beta}}, \text{ se } x \geq 0 \\ 0, \text{ caso contrário} \end{cases}$$

Para $\beta = 1$, a distribuição se reduz à distribuição exponencial.

$$\mu = \alpha^{-\frac{1}{\beta}} \cdot \Gamma\left(1 + \frac{1}{\beta}\right)$$

$$\sigma^2 = \alpha^{-\frac{2}{\beta}} \left\{ \Gamma\left(1 + \frac{2}{\beta}\right) - \left[\Gamma\left(1 + \frac{1}{\beta}\right)\right]^2 \right\}$$

Onde:

$$\Gamma(n) = \int_0^{\infty} t^{n-1} e^{-t} dt$$

$$\Gamma(n) = (n-1)!, n \in \mathbb{N}^*$$

Em particular:

$$\Gamma\left(\frac{3}{2}\right) = \frac{\sqrt{\pi}}{2}$$

A função de confiabilidade (sobrevivência) é:

$$P(T > t) = e^{-\alpha x^{\beta}}$$

3.3. Notas adicionais

Expansão de termo na esperança

Se $C(X) = (X + 5)^2$ é o custo da função, então $E(C(X)) = E(X^2 + 10X + 25) = E(X^2) + 10E(X) + 25$

Tabela de distribuição normal padrão

1. A tabela fornece apenas áreas superiores ao ponto $P(Z \geq z)$. Assim, se queremos $P(Z < 2)$, devemos utilizar $1 - P(Z \geq 2)$.
2. Para usar a tabela, deve-se padronizar a variável aleatória X:

$$Z = \frac{X - \mu}{\sigma}$$

Z-score em Machine Learning

O Z score é útil para otimizar modelos de machine-learning.

$$Z = \frac{X - \mu}{\sigma}$$

1. Eliminação de diferenças de escala: Se uma featura varia entre 0 e 1 e outra entre 0 e 100.000, o gráfico de erro forma um "vale oval e comprimido". O algoritmo perde muito tempo até encontrar o mínimo. Com dados padronizados, as diferenças de escala somem, e o gráfico fica equilibrado (esférico), agilizando o GD. Isso impede que sinais de alta amplitude dominem o aprendizado do modelo.
2. Detecção de outliers: Valores com $|Z| > 3$ são anomalias que se desviam drasticamente da média da população original, e podem ser identificados.

Nota de exercício de distr. normal

Um protocolo de roteamento distribui pacotes de dados de modo que o tempo de latência siga uma distribuição Normal com desvio-padrão de 10 ms. O administrador precisa garantir que apenas 10% dos pacotes tenham latência inferior a 500 ms.

Qual deve ser o tempo médio configurado no sistema?

1. $\sigma = 10$ ms
2. Na tabela, não se pode consultar simplesmente $f_x(Z) = 0.1$, pois a tabela dá $f_x(X \geq 500)$. Assim, temos que buscar $f_x(X \leq 500) = 1 - f_x(X \geq 500)$:

$$1 - f_x(Z) = 0.1 \rightarrow f_x(Z) = 0.9 \rightarrow Z = -1.28$$

1. $Z = \frac{X-\mu}{\sigma} \rightarrow -1.28 = \frac{500-\mu}{10} \rightarrow \mu = 512.8$

Nota de exercício de distr. normal - Acúmulo de média/variância

Um servidor em nuvem suporta até 500 GB de memória RAM para instâncias virtuais. O consumo de cada instância é Normal, com média 70 GB e variância 100 GB².

Qual é a probabilidade aproximada de 7 instâncias independentes ultrapassarem, em conjunto, o limite de 500 GB?

1. Nova média: $7 \cdot 70$ GB $= 490$ GB
2. Nova variância: $7 \cdot 100\text{GB}^2 \rightarrow \sigma = \sqrt{700} = 26.48$ GB
3. $Z = \frac{500-490}{26.48} = 0.378$
4. $f_x(X \geq 500) = P(Z \geq 0.378) = 0.3527$

4. Variáveis Aleatórias Bidimensionais

Sejam X e Y duas variáveis aleatórias. Quando há interesse na variação conjunta de X e Y, estudamos (X,Y) como uma variável aleatória bidimensional.

Variável aleatória bidimensional discreta

A variável aleatória bidimensional discreta (X, Y) tem f.m.p. conjunta definida por:

$$p_{x,y} = P(X = x, Y = y)$$

Ela é utilizada quando há interesse na variação conjunta de X e Y.

Propriedades:

1. $p_{x,y}(x, y) \geq 0$
2. $\sum_{x_i} \sum_{y_i} p_{x,y}(x_i, y_i) = 1$

Example.

$$X = \begin{cases} 1, \text{ se a peça passa no 1 teste} \\ 0, \text{ se a peça falha no 1 teste} \end{cases}$$

$$Y = \begin{cases} 1, \text{ se a peça passa no 2 teste} \\ 0, \text{ se a peça falha no 2 teste} \end{cases}$$

$p_{x,y}(x, y)$	**Y**	
X	0	1
0	0.1	0.2
1	0.2	0.5

□

Distribuições Marginais

Para uma variável aleatória bidimensional, a probabilidade marginal de X é dada por:

$$p_x(x) = \sum_{y_i} p_{x,y}(x, y)$$

E a probabilidade marginal de y é:

$$p_y(y) = \sum_{x} p_{x,y}(x, y)$$

Variável aleatória bidimensional contínua

A variável aleatória bidimensional contínua (X,Y) tem f.d.p. conjunta dada por $f_{x,y}(x,y)$ e f.d.a $F_{x,y}(x,y) = P(X \leq x, y \leq y)$, satisfazendo:

1. $f_{x,y}(x,y) \geq 0$
2. $\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f_{x,y}(x,y)dydx = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f_{x,y}(x,y)dxdy = 1$
3. $F(x,y)(x,y) = P(X \leq x, Y \leq y) = P(X < x, Y \leq y) = P(X \leq x, Y < y) = P(X < x, Y < y)$

Propriedades:

1. $f_{x,y}(x,y) = \frac{\partial F_{x,y}(x,y)}{\partial x \partial y} = \frac{\partial F_{x,y}(x,y)}{\partial y \partial x}$
2. $F_{x,y}(x,y) = \int_{-\infty}^{x}\int_{-\infty}^{y} f_{x,y}(t,w)dwdt$
3. $P(a \leq X \leq b, c \leq Y \leq d) = \int_{a}^{b}\int_{c}^{d} f_{x,y}(x,y)dydx$

Todas as integral são invertíveis por Fubini.

Densidades Marginais

Seja (X,Y) a variável aleatória contínua bidimensional com f.d.p. conjunta $f_{x,y}(x,y)$. A f.d.p. marginal de X é dada por:

$$f_x(x) = \int f_{x,y}dy$$

E a f.d.p. marginal de Y é:

$$f_{y(y)} = \int f_{x,y}(x,y)dx$$

Independência Probabilística

Duas variáveis aleatórias X e Y são independentes se a f.m.p. (ou f.d.p.) conjunta de (X,Y) for fatorável no produto das f.m.p. (ou f.d.p.) marginais de de X e Y:

$$p_{x,y}(x,y) = p_x(x) \cdot p_y(y)$$

$$f_{x,y}(x,y) = f_x(x) \cdot f_y(y)$$

Esperança

$$E(h(X,Y)) = \sum_{x_i}\sum_{y_i} h(x,y)p_{x,y}(x,y)$$

Ou

$$E(h(X,Y)) = \int\int h(x,y)f_{x,y}dydx$$

Covariância

$$\sigma_{XY} = Cov(X,Y) = E\big((X-\mu_x)(Y-\mu_y)\big) = E(XY) - E(X)E(Y) = E(XY) - \mu_x\mu_y$$

E a correlação entre X e Y por:

$$\rho_{x,y} = \frac{Cov(X,Y)}{\sqrt{Var(X)}\sqrt{Var(Y)}} = \frac{\sigma_{xy}}{\sigma_x\sigma_y}$$

Independência e correlação

Se X e Y são duas variáveis aleatórias independentes, então a correlação entre X e Y é nula. A volta não vale.

Variância de X e Y

Propriedades:

1. $Var(X+Y) = Var(X) + Var(Y) + 2Cov(X,Y)$
2. $Var(X-Y) = Var(X) + Var(Y) - 2Cov(X,Y)$
3. $Var(aX \pm bY) = a^2Var(X) + b^2Var(Y) \pm 2abCov(X,Y)$
4. Se X,Y independentes: $Cov(X,Y) = 0$

Combinação linear de variáveis aleatórias normais independentes

Sejam $X_1, X_2, ..., X_n$ n variáveis aleatórias independentes tais que $X_i \sim N(\mu, \sigma^2)$.

Considere $Y = \sum_{i=1}^{n} X_i$. Temos que:

$$E(Y) = E\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} E(X_i) = \sum_{i=1}^{n} \mu = n\mu$$

e

$$Var(Y) = Var\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} Var(X_i) = \sum_{i=1}^{n} \sigma^2 = n\sigma^2$$

Padronizando:

$$\frac{Y - E(Y)}{\sqrt{Var(Y)}} = \frac{Y - n\mu}{\mu\sqrt{n}} \sim N(0, 1)$$